

Core Java Problem-Solving & Notes

A definitive, publication-grade reference manual featuring detailed theoretical definitions, flowcharts, production Java implementations, and edge-case analysis.



157 Problem Files thoroughly documented



Vector SVG Flowcharts for logic visualization



17 Structured Modules from syntax to concurrency



Time & Space Complexity asymptotic profiles

Core Java Learning Journey

Prepared for Academic Excellence & Technical Interviews

Repository: [Core-java](#) • [kamraj-college-webdev](#)

17

MODULES

157

JAVA FILES

100%

COVERAGE

Table of Contents & Study Guide

Systematic overview of all 17 Core Java learning modules. Every module contains rigorous definitions, flowcharts, complete Java source code, walkthrough steps, sample I/O, and critical boundary edge cases.

MODULE 1 7 In-Depth Topics Basic Syntax, Data Types & Conditionals Foundational Java constructs: primitive data types, relational operators, ternary logic, conditional branching (if-else, switch-case), and validation workflows.	MODULE 2 5 In-Depth Topics Iteration, Control Structures & Digit Extraction Loop constructs (while, for, do-while), accumulator patterns, sentinel-controlled loops, integer arithmetic decomposition, and iterative state tracking.
MODULE 3 2 In-Depth Topics Functions, Methods & Method Overloading Modular functional decomposition, parameter passing, return semantics, stack frames, and compile-time polymorphism via method overloading.	MODULE 4 3 In-Depth Topics Object-Oriented Programming (OOP) Fundamentals Class blueprints, object instantiation, constructor initialization, state encapsulation with private fields, domain state machines, and object arrays.
MODULE 5 3 In-Depth Topics Inheritance, Polymorphism & Abstraction IS-A relationships ('extends'), method overriding, runtime dynamic method dispatch ('super'), abstract classes vs pure interfaces, and multiple interface implementation.	MODULE 6 5 In-Depth Topics 1D Array Algorithms: Searching, Extremes & Manipulations Contiguous memory structures, index arithmetic, linear searching, single-pass extreme tracking, two-pointer swapping, in-place element partition, and cyclic rotations.
MODULE 7 3 In-Depth Topics 2D Arrays & Matrix Mathematics Row-major multidimensional grids, nested coordinate iterations, matrix algebra (addition, dot product multiplication, transposition), diagonal traversal, and cell search.	MODULE 8 4 In-Depth Topics String Processing & Text Parsing Algorithms Immutability of Java Strings, String pool internals, character tokenization, palindrome verification, anagram hashing, prefix overlap, and frequency maps.
MODULE 9 3 In-Depth Topics Recursion & Divide-and-Conquer Call stack frames, base case termination, recurrence relations, linked list recursive inversion, and combinatorial power set branch exploration.	MODULE 10 3 In-Depth Topics Advanced Algorithmic Paradigms (Two Pointers & Sliding Window) Optimizing brute force $O(N^2)$ to linear $O(N)$ using two-pointer convergence, target difference hunting, and fixed/dynamic sliding window accumulators.
MODULE 11 1 In-Depth Topics Geometric CLI Pattern Printing Nested loop coordinate geometry, row-column mapping, leading whitespace indentation formulas, and CLI shape rendering.	MODULE 12 2 In-Depth Topics Exception Handling & Robust System Architecture Throwable hierarchy, Checked vs Unchecked exceptions, try-catch-finally mechanics, resource cleanup, and domain-specific custom exception classes.
MODULE 13 2 In-Depth Topics	MODULE 14 2 In-Depth Topics

Collections Framework (List, Set, Map)

Dynamic resizable arrays (ArrayList), hashing-based sets with duplicate rejection (HashSet), associative key-value lookup tables (HashMap), and the Student Management System.

Object Sorting, Comparable & Comparator

Natural ordering with java.lang.Comparable (compareTo), flexible multi-attribute sorting strategies via java.util.Comparator, and the Employee Sorting System.

MODULE 15

1 In-Depth Topics

Java Generics Deep Dive

Parameterized types (), compile-time type safety, erasure of casting boilerplate, generic methods, and coordinated multi-collection architecture.

MODULE 16

1 In-Depth Topics

Java File Handling & Persistent I/O

Disk I/O streams, File existence checks, character streams (FileWriter, FileReader), buffering with BufferedReader, append modes, and file-persisted databases.

MODULE 17

3 In-Depth Topics

Modern Date & Time API (java.time) & Multithreading

Immutable temporal classes (LocalDate, Period, ChronoUnit), custom formatting (DateTimeFormatter), thread lifecycle concurrency (Thread, Runnable), and the Employee Leave Management System.

Basic Syntax, Data Types & Conditionals

Even or Odd Parity Determination

Parity /
Arithmetic

Time:
O(1)

Space:
O(1)

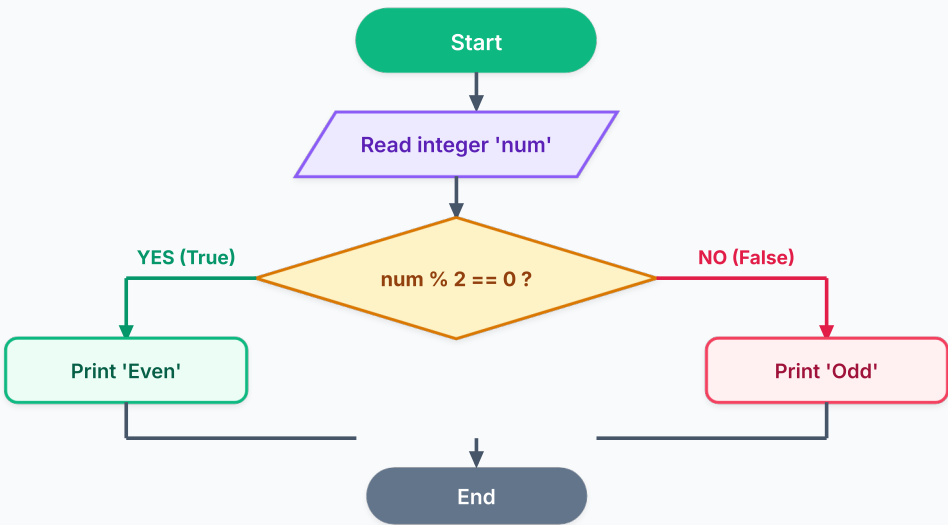
Files: [CheckEvenorOdd.java](#) • [CheckEvenOrOddMethod.java](#) • [EvenorOddMethod.java](#)

DETAILED DEFINITION & CORE CONCEPT

In number theory, parity refers to the attribute of an integer being either even or odd. An integer n is **even** if it is an exact multiple of 2 (i.e., $n = 2k$ for some integer k), and **odd** if it leaves a remainder of 1 or -1 when divided by 2.

In Java, parity is calculated using the remainder operator `%` (modulo). Alternatively, it can be tested at the bitwise level via `(n & 1) == 0`, checking the least significant bit (LSB), which is 0 for even numbers and 1 for odd numbers.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CheckEvenorOdd.java

JAVA 17+

```
import java.util.Scanner;

public class CheckEvenorOdd {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        // Ternary operator evaluation of parity
        String result = (num % 2 == 0) ? "Even" : "Odd";
        System.out.println("The number is: " + result);
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Read an integer input from the user via `Scanner.nextInt()`.
2. Evaluate condition `num % 2 == 0`: if true, value evaluates to 'Even'; otherwise 'Odd'.
3. Print the computed string to the standard output console.

SAMPLE INPUT

Enter a number: 14

EXPECTED OUTPUT

The number is: Even

Critical Edge Cases & Pitfalls

Zero: `0 % 2 == 0`, properly identified as Even.

Negative Numbers: In Java, `-3 % 2` evaluates to `-1`. Checking `num % 2 != 0` correctly flags all odd integers regardless of algebraic sign.

Sign Classification (Positive, Negative, Zero)

Conditionals / Multi-Branch

Time:
O(1)

Space:
O(1)

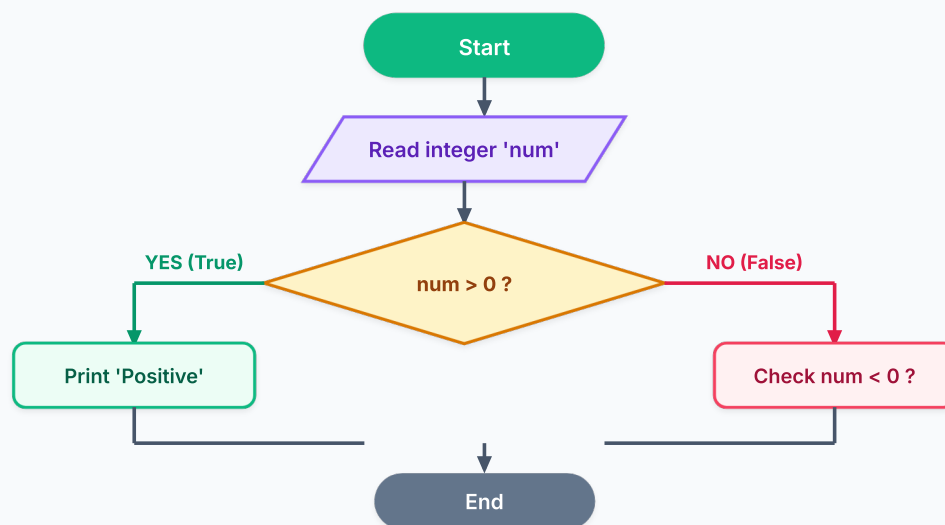
Files: CheckPostiveorNegative.java •
CountPositiveNegativeZero.java

DETAILED DEFINITION & CORE CONCEPT

Classification of real numbers based on their position relative to the origin on the number line. A value strictly greater than 0 is **positive**, strictly less than 0 is **negative**, and equal to 0 is **neutral (zero)**.

Multi-branch conditional structures (if-else if-else) provide mutually exclusive evaluations ensuring exactly one branch executes in constant time $O(1)$.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CheckPostiveorNegative.java

JAVA 17+

```
import java.util.Scanner;

public class CheckPostiveorNegative {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        if (num > 0) {
            System.out.println("The number is Positive.");
        } else if (num < 0) {
            System.out.println("The number is Negative.");
        } else {
            System.out.println("The number is Zero.");
        }
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Input an integer num.

2. First test if `num > 0`. If true, branch to Positive.
3. Else test if `num < 0`. If true, branch to Negative.
4. If neither condition holds, `num` is 0 by mathematical trichotomy.

SAMPLE INPUT

Enter a number: -42

EXPECTED OUTPUT

The number is Negative.

Critical Edge Cases & Pitfalls

Input Zero: Evaluates both guards as false and correctly lands in the catch-all `else` block.

Boundary Extremes: Handled accurately for `Integer.MIN_VALUE` (-2147483648) and `Integer.MAX_VALUE` (2147483647).

Largest Among Three Numbers

Relational Logic /
Selection

Time:
O(1)

Space:
O(1)

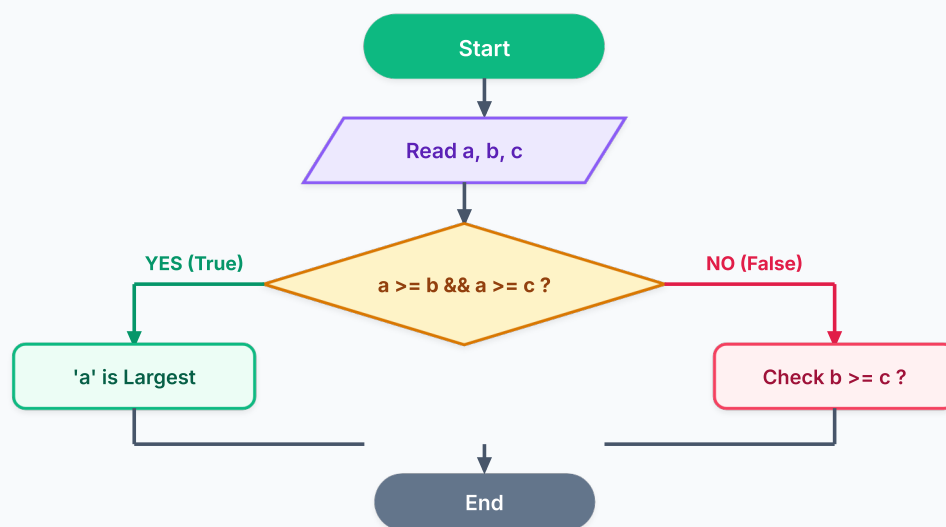
Files: [LargestofThree.java](#) • [LargestofThreeNumbers.java](#) • [Largest.java](#)

DETAILED DEFINITION & CORE CONCEPT

Finding the mathematical maximum element among three numerical inputs \$a, b, c\$. This problem tests compound relational logic with logical AND (&&) or nested comparative guards.

Using short-circuit evaluation in Java, `(a >= b && a >= c)` ensures that if the first sub-expression evaluates to false, the second is never evaluated, optimizing instruction cycles.

ALGORITHMIC LOGIC & DECISION FLOWCHART



LargestofThree.java

JAVA 17+

```
import java.util.Scanner;

public class LargestofThree {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter three numbers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();

        if (a >= b && a >= c) {
            System.out.println(a + " is the largest.");
        } else if (b >= c) {
            System.out.println(b + " is the largest.");
        } else {
            System.out.println(c + " is the largest.");
        }
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Accept three numerical inputs `a`, `b`, `c`.

2. Compare a against both b and c.
3. If a is not the maximum, compare b with c.
4. Output the determined maximum value.

SAMPLE INPUT

Enter three numbers: 45 89 12

EXPECTED OUTPUT

89 is the largest.

Critical Edge Cases & Pitfalls

Identical Elements: Using `>=` instead of `>` prevents errors when two or three numbers are equal (e.g., 10, 10, 5).

Academic Grade Categorization

Tiered Conditionals

Time: O(1)

Space: O(1)

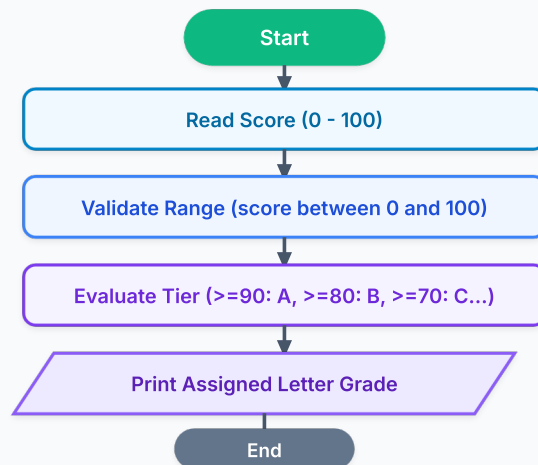
Files: GradeCalculator.java

DETAILED DEFINITION & CORE CONCEPT

Mapping continuous numerical percentages (0-100) into discrete ordinal grade tiers (A, B, C, D, Fail).

Structured as a sequential falling-edge comparison: because previous intervals are progressively filtered out, each check only needs a lower-bound comparison (e.g., marks ≥ 90 , then marks ≥ 80).

ALGORITHMIC LOGIC & DECISION FLOWCHART



GradeCalculator.java

JAVA 17+

```
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter student marks (0-100): ");
        int marks = sc.nextInt();

        if (marks < 0 || marks > 100) {
            System.out.println("Invalid Marks! Must be 0-100.");
        } else if (marks >= 90) {
            System.out.println("Grade: A+");
        } else if (marks >= 80) {
            System.out.println("Grade: A");
        } else if (marks >= 70) {
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
        } else if (marks >= 50) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: Fail");
        }
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Input score integer.
2. Verify input validity $0 \leq \text{marks} \leq 100$.
3. Sequentially descend through threshold brackets.
4. Print assigned letter grade.

SAMPLE INPUT

Enter student marks (0-100): 86

EXPECTED OUTPUT

Grade: A

Critical Edge Cases & Pitfalls

Out of Bounds: Inputs like 105 or -10 are intercepted by the primary validation guard.

Slab-Based Tariff Electricity Calculation

Progressive Tier
Billing

Time:
O(1)

Space:
O(1)

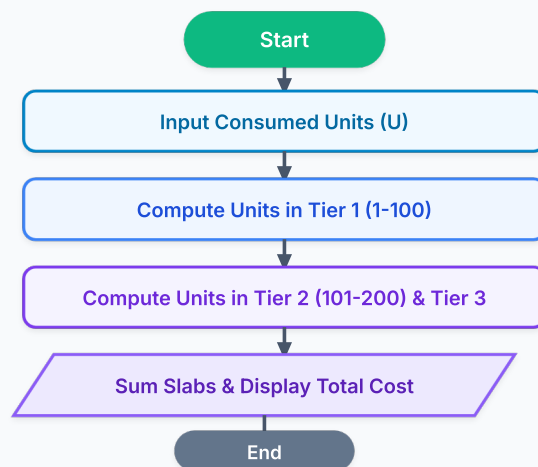
Files: ElectricityBill.java

DETAILED DEFINITION & CORE CONCEPT

Progressive tiered billing calculates cost by applying incremental rate tariffs to specific unit slabs rather than multiplying the entire consumption by a flat rate.

Standard slabs: First 100 units @ \$1.5/unit, next 100 units (101-200) @ \$2.5/unit, next 100 units (201-300) @ \$4.0/unit, and units above 300 @ \$5.0/unit.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ElectricityBill.java

JAVA 17+

```
import java.util.Scanner;

public class ElectricityBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter units consumed: ");
        int units = sc.nextInt();
        double bill = 0;

        if (units <= 100) {
            bill = units * 1.5;
        } else if (units <= 200) {
            bill = (100 * 1.5) + (units - 100) * 2.5;
        } else if (units <= 300) {
            bill = (100 * 1.5) + (100 * 2.5) + (units - 200) * 4.0;
        } else {
            bill = (100 * 1.5) + (100 * 2.5) + (100 * 4.0) + (units - 300) * 5.0;
        }

        System.out.println("Total Electricity Bill: $" + bill);
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Read consumed electricity units.
2. Determine the bracket range.
3. Accumulate fixed slab charges for completed tiers and calculate the remainder rate for the top tier.
4. Display calculated total.

SAMPLE INPUT

Enter units consumed: 250

EXPECTED OUTPUT

Total Electricity Bill: \$600.0

Critical Edge Cases & Pitfalls

Zero consumption: units = 0 correctly results in \$0.0.

Boundary Units: Exactly 100, 200, 300 transition seamlessly between tiers without double-counting.

Temporal Switch-Case Mapping (Days & Months)

Jump Table /
Switch-Case

Time:
O(1)

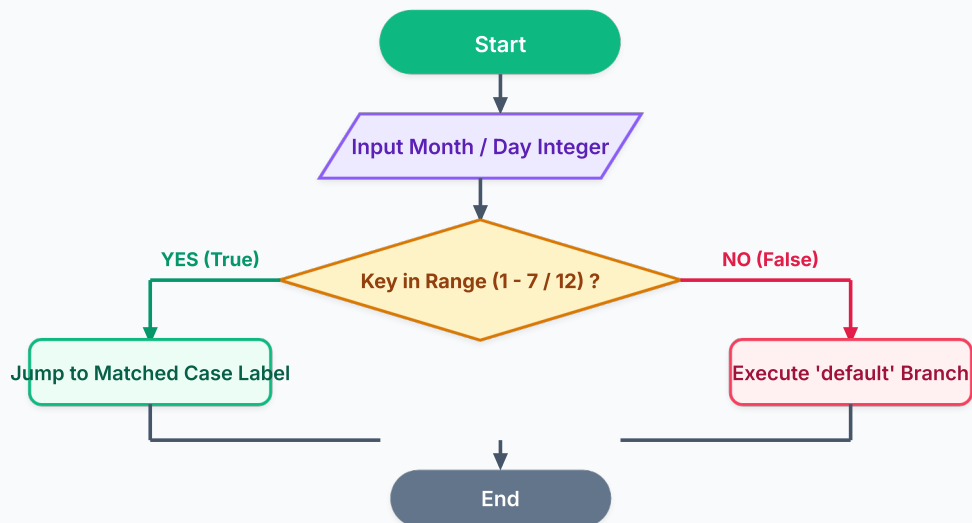
Space:
O(1)

Files: [DayOfTheWeek.java](#) • [Monthname.java](#)

DETAILED DEFINITION & CORE CONCEPT

Multi-way selection mapping an integer key (1-7 or 1-12) to a string representation. The Java switch statement translates into a `tableswitch` or `lookupswitch` bytecode instruction, allowing fast constant-time branching.

ALGORITHMIC LOGIC & DECISION FLOWCHART



DayOfTheWeek.java

JAVA 17+

```
import java.util.Scanner;

public class DayOfTheWeek {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter day number (1-7): ");
        int day = sc.nextInt();

        switch (day) {
            case 1 -> System.out.println("Monday");
            case 2 -> System.out.println("Tuesday");
            case 3 -> System.out.println("Wednesday");
            case 4 -> System.out.println("Thursday");
            case 5 -> System.out.println("Friday");
            case 6 -> System.out.println("Saturday");
            case 7 -> System.out.println("Sunday");
            default -> System.out.println("Invalid day! Enter 1-7.");
        }
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Accept integer choice (1 to 7).

2. Evaluate switch expression.
3. Jump directly to corresponding case and print name.
4. Default branch executes if input is out of valid range.

SAMPLE INPUT

Enter day number (1-7): 5

EXPECTED OUTPUT

Friday

Critical Edge Cases & Pitfalls

Fall-through Prevention: Using enhanced arrow switch (->) or explicit break; statements prevents accidental execution of successive case branches.

String Credential Authentication

String Comparison / Security

Time: O(1)

Space: O(1)

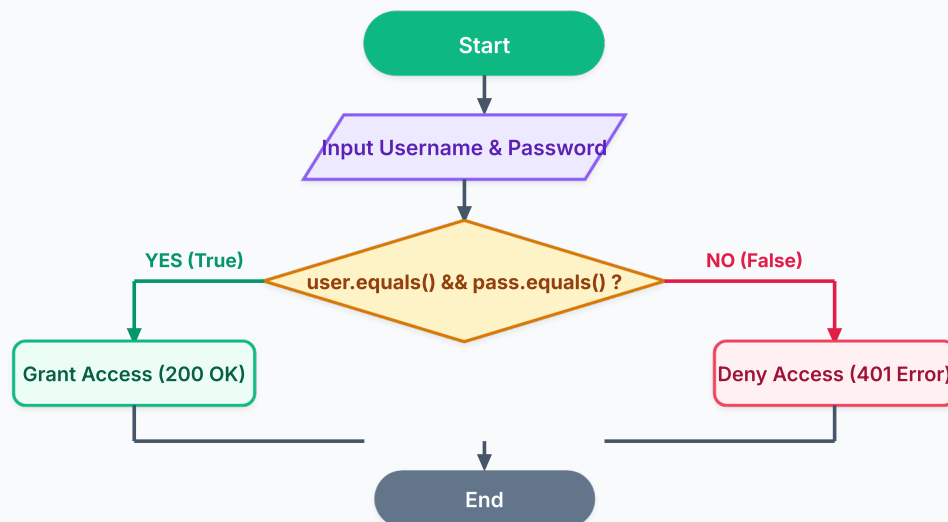
Files: Login.java

DETAILED DEFINITION & CORE CONCEPT

Validating user authentication credentials against stored reference values.

Crucial Java Core Concept: In Java, Strings are reference types. Comparing Strings using `==` evaluates reference identity (memory addresses in the String pool), NOT character content. Value equality must always be performed using `str.equals()`.

ALGORITHMIC LOGIC & DECISION FLOWCHART



Login.java

JAVA 17+

```
import java.util.Scanner;

public class Login {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Username: ");
        String username = sc.nextLine();
        System.out.print("Password: ");
        String password = sc.nextLine();

        // Strict value comparison using .equals()
        if ("admin".equals(username) && "secure123".equals(password)) {
            System.out.println("Login Successful! Welcome.");
        } else {
            System.out.println("Access Denied: Invalid Credentials.");
        }
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Read username and password strings.
2. Perform content equality comparison via `"admin".equals(username)` (Yoda condition to prevent `NullPointerException`).

3. Print authentication confirmation or rejection message.

SAMPLE INPUT

Username: admin\nPassword: secure123

EXPECTED OUTPUT

Login Successful! Welcome.

Critical Edge Cases & Pitfalls

Null Pointer Defense: Putting the string literal first (`"admin".equals(input)`) prevents crashes even if `input` is null.

MODULE 2 • CORE JAVA CURRICULUM

Iteration, Control Structures & Digit Extraction

Sequence Generation & Accumulation (1 to N)

Iteration / Loop
Mechanics

Time:
 $O(N)$

Space:
 $O(1)$

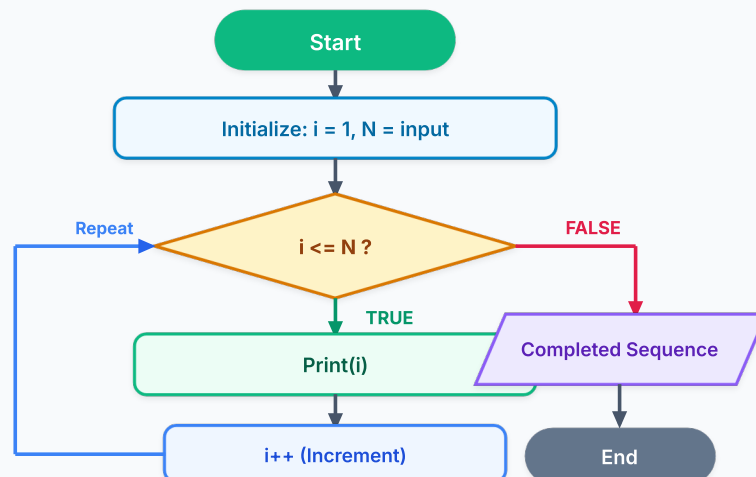
Files: [OnetoNWhile.java](#) • [UptoN.java](#) • [EvenNumbers.java](#) • [Count.java](#)

DETAILED DEFINITION & CORE CONCEPT

Fundamental iterative progression generating sequential integers from 1 up to a positive threshold N .

Demonstrates the three pillars of all loops: 1. **Initialization** (`int i = 1`) 2. **Termination Condition** (`i <= N`) 3. **State Progression / Step** (`i++` or `i += 2` for evens).

ALGORITHMIC LOGIC & DECISION FLOWCHART



OnetoNWhile.java

JAVA 17+

```
import java.util.Scanner;

public class OnetoNWhile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter N: ");
        int n = sc.nextInt();

        int i = 1;
        while (i <= n) {
            System.out.print(i + " ");
            i++;
        }
        System.out.println();
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Input limit N .
2. Initialize loop index $i = 1$.
3. Verify guard condition $i \leq n$. If true, print i and increment.
4. When $i > n$, loop terminates.

SAMPLE INPUT

Enter N: 5

EXPECTED OUTPUT

1 2 3 4 5

Critical Edge Cases & Pitfalls

$N \leq 0$: While loop condition is immediately false; loop body never executes (safe zero-iteration).

Sentinel-Controlled Accumulation

Sentinel Loops / Accumulator

Time: $O(K)$ where K is number of entries

Space: $O(1)$

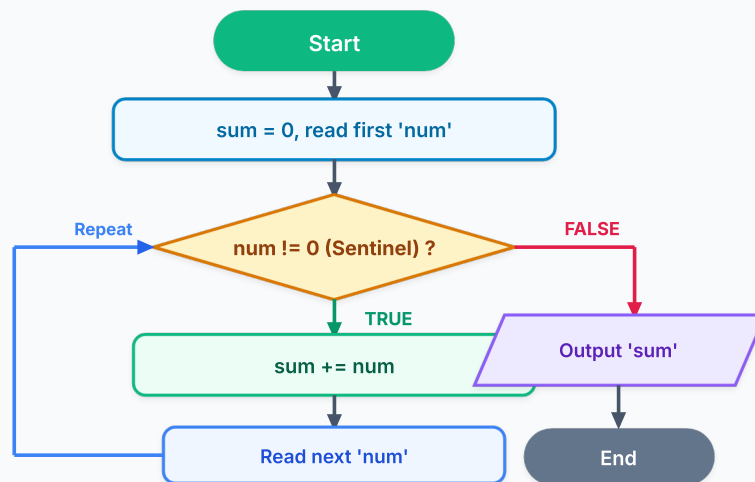
Files: [SumUntilZero.java](#) • [SumEvenOdd.java](#) • [Sum.java](#) • [Multiply.java](#)

DETAILED DEFINITION & CORE CONCEPT

In many real-world input streams, the total number of inputs is unknown in advance. A **sentinel value** (such as 0) acts as an explicit termination signal.

The loop continuously ingests user inputs and updates an accumulator register ($\text{sum} += \text{num}$) until the sentinel value is encountered.

ALGORITHMIC LOGIC & DECISION FLOWCHART



SumUntilZero.java

JAVA 17+

```
import java.util.Scanner;

public class SumUntilZero {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int sum = 0;
        int num;

        System.out.println("Enter numbers to sum (enter 0 to stop):");
        while ((num = sc.nextInt()) != 0) {
            sum += num;
        }

        System.out.println("Final Accumulated Sum = " + sum);
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize accumulator variable $\text{sum} = 0$.
2. Read input into num in the while condition check.

3. If num != 0, add to sum and repeat.
4. Once 0 is entered, terminate loop and display final total.

SAMPLE INPUT

10\n25\n-5\n0

EXPECTED OUTPUT

Final Accumulated Sum = 30

Critical Edge Cases & Pitfalls

Immediate Sentinel: Entering 0 on the very first prompt results in sum = 0 without error.

Digit Extraction, Counting & Summation

Arithmetic
Decomposition

Time:
 $O(\log_{10} N)$

Space:
 $O(1)$

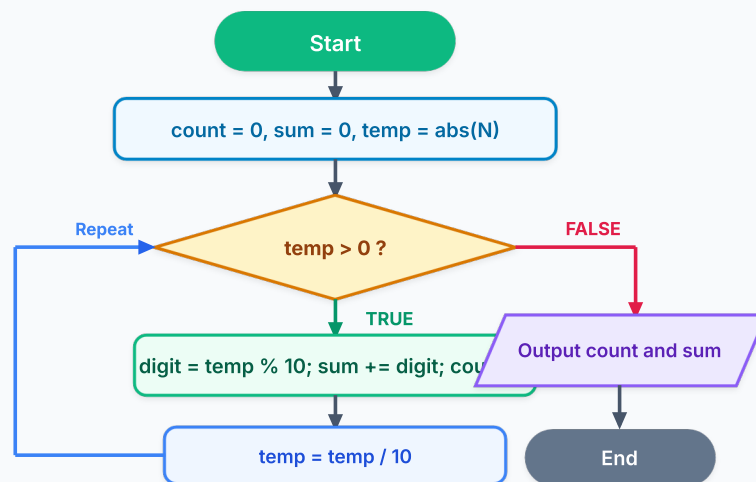
Files: [CountDigits.java](#) • [CountSumDigits.java](#) • [ReverseNumber.java](#)

DETAILED DEFINITION & CORE CONCEPT

Extracting the decimal digits of an integer relies on radix-10 arithmetic: 1. **Extract LSB (last digit):** $\text{digit} = \text{num} \% 10$ 2. **Truncate LSB:** $\text{num} = \text{num} / 10$

The number of iterations is proportional to the number of decimal digits: $\lfloor \log_{10}(N) \rfloor + 1$.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CountDigits.java

JAVA 17+

```
import java.util.Scanner;

public class CountSumDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int n = sc.nextInt();

        int temp = Math.abs(n);
        int count = (temp == 0) ? 1 : 0;
        int sum = 0;

        while (temp > 0) {
            int digit = temp % 10;
            sum += digit;
            count++;
            temp /= 10;
        }

        System.out.println("Total Digits: " + count);
        System.out.println("Sum of Digits: " + sum);
        sc.close();
    }
}
```

```
}  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Take absolute value of input `n` to handle negative values.
2. Extract rightmost digit via `temp % 10`.
3. Add extracted digit to sum and increment count.
4. Integer-divide `temp` by 10 to shift digits right.
5. Repeat until `temp == 0`.

SAMPLE INPUT

Enter an integer: 54321

EXPECTED OUTPUT

Total Digits: 5\nSum of Digits: 15

Critical Edge Cases & Pitfalls

Input 0: Mathematically has 1 digit with sum 0. The ternary operator `(temp == 0) ? 1 : 0` gracefully prevents reporting 0 digits.

Integer Reversal via Horner's Accumulator

Arithmetic
Transformations

Time: $O(\log_{10} N)$

Space:
 $O(1)$

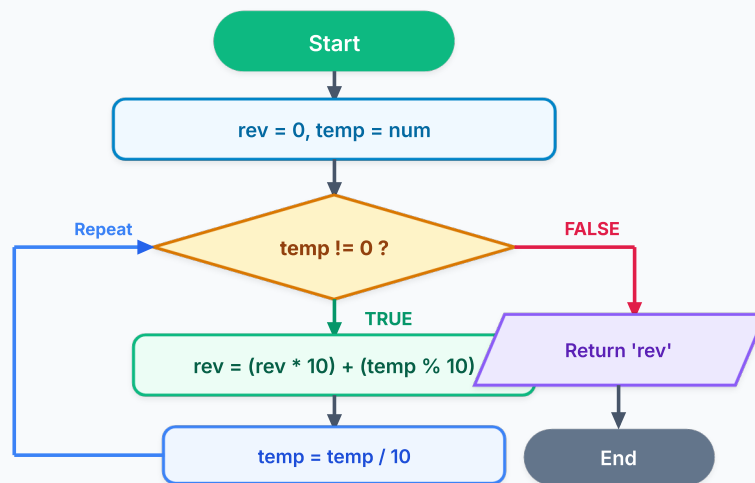
Files: ReverseNumber.java

DETAILED DEFINITION & CORE CONCEPT

Reversing an integer mathematically without converting it to a string. Uses an accumulator initialized to 0. In each step, the existing reversed value is multiplied by 10 (shifting left) and the extracted digit is added:

$$\text{rev} = (\text{rev} \times 10) + (n \% 10)$$

ALGORITHMIC LOGIC & DECISION FLOWCHART



ReverseNumber.java

JAVA 17+

```
import java.util.Scanner;

public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = sc.nextInt();

        int rev = 0;
        while (num != 0) {
            int digit = num % 10;
            rev = (rev * 10) + digit;
            num /= 10;
        }

        System.out.println("Reversed Number = " + rev);
        sc.close();
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize `rev = 0`.
2. Extract last digit: `digit = num % 10`.
3. Shift previous reversed digits left by multiplying by 10 and add the current digit.

4. Divide num by 10.
5. Terminate when num == 0.

SAMPLE INPUT

Enter number: 12345

EXPECTED OUTPUT

Reversed Number = 54321

Critical Edge Cases & Pitfalls

Trailing Zeroes: An input like 1200 will output 21 (numerical value, since leading zeros are dropped in mathematical integers).

CLI Binary Search Number Guessing Game

Interactive Loops /
Randomization

Time: $O(\log N)$ optimal
attempts

Space:
 $O(1)$

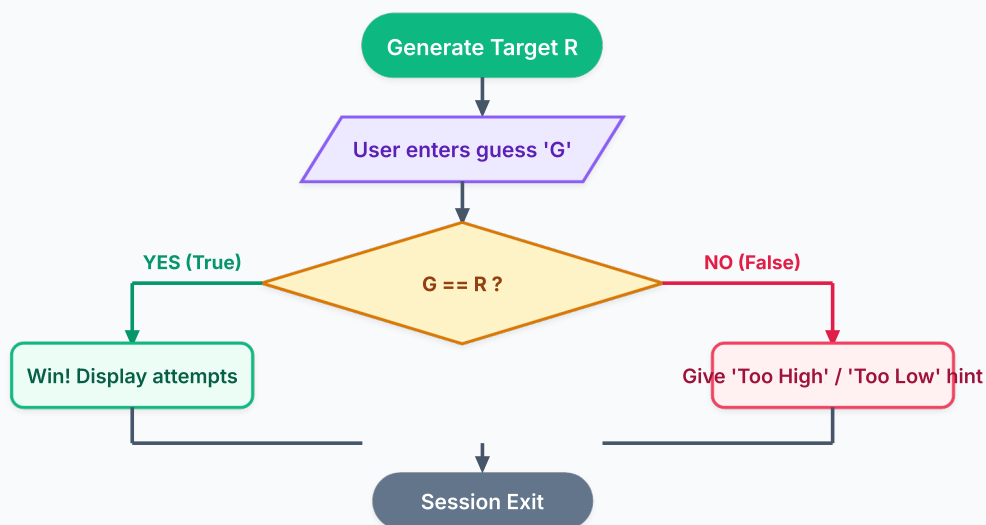
Files: NumberGuessing.java

DETAILED DEFINITION & CORE CONCEPT

An interactive feedback game demonstrating loop control flow and conditional narrowing. The computer generates a pseudorandom integer R in $[1, 100]$. With each user guess, feedback guides the player: - Guess < Target: "Too Low!" - Guess > Target: "Too High!" - Guess == Target: "Correct! Game Won."

Using optimal binary search strategy, the secret number can always be discovered in $\lceil \log_2(100) \rceil = 7$ attempts.

ALGORITHMIC LOGIC & DECISION FLOWCHART



NumberGuessing.java

JAVA 17+

```
import java.util.Scanner;
import java.util.Random;

public class NumberGuessing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random rand = new Random();
        int target = rand.nextInt(100) + 1; // 1 to 100
        int attempts = 0;
        int guess;

        System.out.println("I have chosen a number between 1 and 100. Guess it!");
        do {
            System.out.print("Your guess: ");
            guess = sc.nextInt();
            attempts++;

            if (guess > target) {
                System.out.println("Too High! Try again.");
            } else if (guess < target) {
                System.out.println("Too Low! Try again.");
            } else {
                System.out.println("Correct! Game Won.");
            }
        } while (true);

        System.out.println("Game ended. Attempts: " + attempts);
    }
}
```

```

        System.out.println("Bravo! You guessed it in " + attempts + " attempts.");
    }
} while (guess != target);

sc.close();
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Generate random target integer in range [1, 100].
2. Execute do-while loop, guaranteeing at least one prompt.
3. Compare user guess against target.
4. Display directional guidance until guess matches target.

SAMPLE INPUT

Your guess: 50 → Too Low!\nYour guess: 75 → Too High!\nYour guess: 62 → Bravo!

EXPECTED OUTPUT

Bravo! You guessed it in 3 attempts.

Critical Edge Cases & Pitfalls

do-while vs while: A do-while loop ensures the guess prompt executes before evaluating the condition, guaranteeing the first guess is read.

MODULE 3 • CORE JAVA CURRICULUM

Functions, Methods & Method Overloading

Modular Functional Calculator

Methods &
Modular
Decomposition

Time:
O(1)

Space:
O(1)

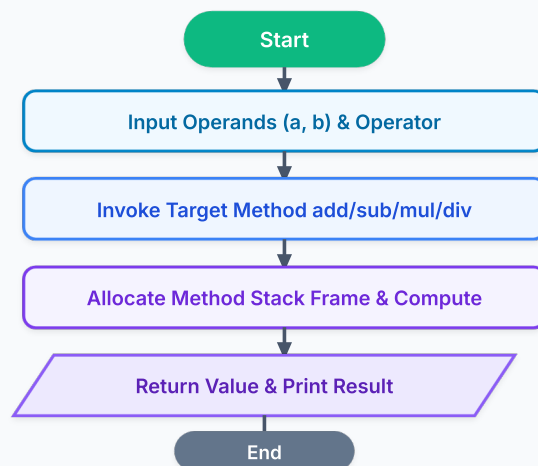
Files: [CalculatorUsingMethod.java](#) • [CalculatorUsingMethods.java](#) • [Calculator.java](#) • [SquareofaNumber.java](#)

DETAILED DEFINITION & CORE CONCEPT

Decomposition breaks a complex program into small, independent, reusable functions. Each method adheres to the **Single Responsibility Principle (SRP)**: - add(a, b) - subtract(a, b) - multiply(a, b) - divide(a, b)

Methods are executed inside isolated stack frames allocated upon invocation and deallocated upon return.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CalculatorUsingMethod.java

JAVA 17+

```
public class CalculatorUsingMethod {  
    public static double add(double a, double b) { return a + b; }  
    public static double subtract(double a, double b) { return a - b; }  
    public static double multiply(double a, double b) { return a * b; }  
    public static double divide(double a, double b) {  
        if (b == 0) {  
            System.out.println("Error: Division by zero!");  
            return Double.NaN;  
        }  
        return a / b;  
    }  
  
    public static void main(String[] args) {  
        double x = 20, y = 4;  
        System.out.println("Add: " + add(x, y));  
        System.out.println("Sub: " + subtract(x, y));  
        System.out.println("Mul: " + multiply(x, y));  
        System.out.println("Div: " + divide(x, y));  
    }  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Define modular methods with `public static` modifiers.
2. Pass parameters by value onto the call stack.
3. Perform arithmetic computation inside the method body.
4. Return the computed result back to the caller frame.

SAMPLE INPUT

Operands: 20, 4

EXPECTED OUTPUT

Add: 24.0\nSub: 16.0\nMul: 80.0\nDiv: 5.0

Critical Edge Cases & Pitfalls

Division by Zero: Guarded using `if (b == 0)` returning `Double.NaN` to prevent illegal arithmetic.

Compile-Time Polymorphism (Method Overloading)

Polymorphism /
Signatures

Time:
O(1)

Space:
O(1)

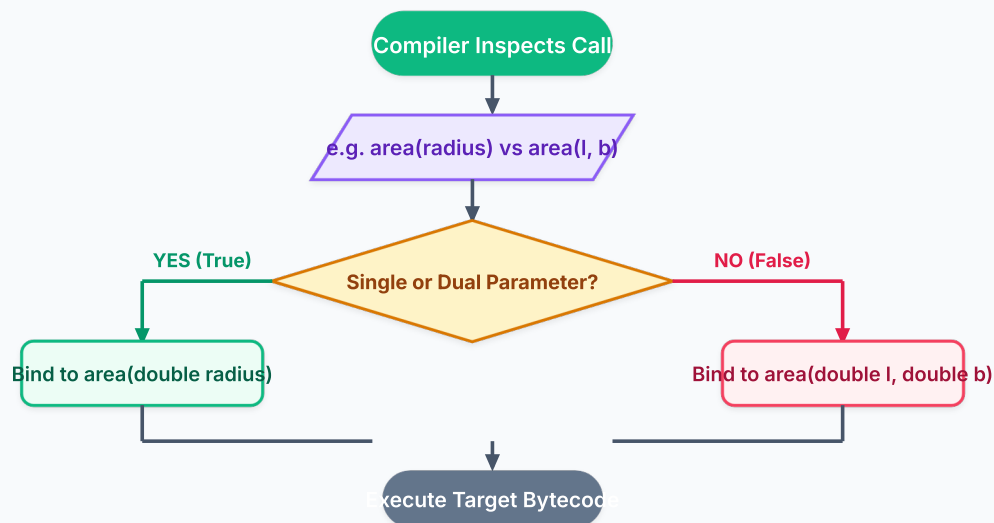
Files: [Overloadedadd.java](#) • [Overloadedarea.java](#) • [greet.java](#)

DETAILED DEFINITION & CORE CONCEPT

Method overloading allows a class to define multiple methods with the exact same name, provided their parameter lists differ in: 1. **Number of parameters** (e.g., `add(int, int)` vs `add(int, int, int)`) 2. **Data types of parameters** (e.g., `add(int, int)` vs `add(double, double)`) 3. **Sequence of parameter types**.

Note: Overloading is resolved strictly at compile time based on parameter types. Varying the return type alone does NOT constitute a valid overload.

ALGORITHMIC LOGIC & DECISION FLOWCHART



Overloadedadd.java

JAVA 17+

```
public class Overloadedarea {
    // Circle Area: single argument
    public static double area(double radius) {
        return Math.PI * radius * radius;
    }

    // Rectangle Area: dual arguments
    public static double area(double length, double width) {
        return length * width;
    }

    // Triangle Area: three arguments (base, height, shape flag)
    public static double area(double base, double height, boolean isTriangle) {
        return 0.5 * base * height;
    }

    public static void main(String[] args) {
        System.out.printf("Circle Area (r=5): %.2f\n", area(5.0));
        System.out.printf("Rectangle Area (4x6): %.2f\n", area(4.0, 6.0));
        System.out.printf("Triangle Area (b=3, h=8): %.2f\n", area(3.0, 8.0, true));
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Define overloaded methods with different signatures.
2. Compiler inspects argument types at the call site.
3. Directly binds call to appropriate method signature in the generated bytecode.

SAMPLE INPUT

Calls: area(5.0), area(4.0, 6.0)

EXPECTED OUTPUT

Circle Area (r=5): 78.54\nRectangle Area (4x6): 24.00\nTriangle Area (b=3, h=8): 12.00

Critical Edge Cases & Pitfalls

Type Promotion: If an exact match is absent, Java automatically promotes primitives (e.g. byte -> short -> int -> long -> float -> double).

MODULE 4 • CORE JAVA CURRICULUM

Object-Oriented Programming (OOP) Fundamentals

Class blueprints, object instantiation, constructor initialization, state encapsulation with private fields, domain state machines, and object arrays.

Class Blueprints & State Encapsulation

OOP Core /
Encapsulation

Time:
O(1)

Space: O(1)
per object

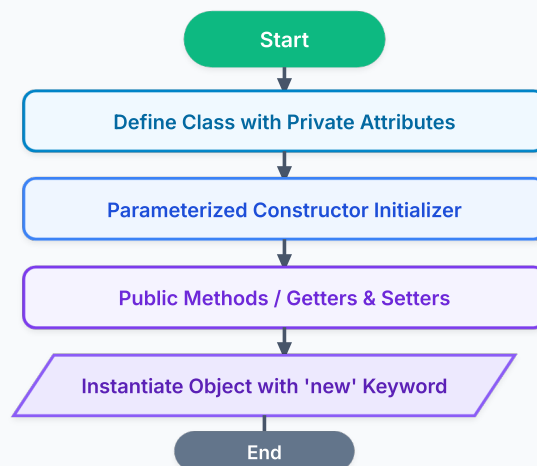
Files: [CreateaCarClass.java](#) • [CreateaStudentClass.java](#) • [RectangleClass.java](#) • [ProductClass.java](#)

DETAILED DEFINITION & CORE CONCEPT

Encapsulation bundles state (instance variables) and behavior (methods) together into a single unit (class), while hiding internal representations using access modifiers.

Fields are marked private to prevent unauthorized external mutation. Controlled read/write access is mediated via public getters and setters with internal validation invariants.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CreateaCarClass.java

JAVA 17+

```
public class CreateaStudentClass {
    static class Student {
        private int id;
        private String name;
        private double marks;

        public Student(int id, String name, double marks) {
            this.id = id;
            this.name = name;
            this.setMarks(marks);
        }

        public void setMarks(double marks) {
            if (marks >= 0 && marks <= 100) {
                this.marks = marks;
            } else {
                System.out.println("Invalid marks!");
            }
        }

        public void display() {
            System.out.println("ID: " + id + " | Name: " + name + " | Marks: " + marks);
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    Student s1 = new Student(101, "Alice", 92.5);
    Student s2 = new Student(102, "Bob", 78.0);
    s1.display();
    s2.display();
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Declare private member variables `id`, `name`, `marks`.
2. Constructor uses `this` keyword to disambiguate parameter names from field names.
3. Setter validates marks are within valid range `[0, 100]`.
4. Instantiate objects in heap memory using `new Student(...)`.

SAMPLE INPUT

Students: (101, Alice, 92.5), (102, Bob, 78.0)

EXPECTED OUTPUT

ID: 101 | Name: Alice | Marks: 92.5\nID: 102 |
Name: Bob | Marks: 78.0

Critical Edge Cases & Pitfalls

Shadowing Protection: Forgetting `this.name = name` results in local parameter self-assignment, leaving instance fields at default values (null/0).

Bank Account Transaction State Machine

Files: [BankAccountClass.java](#) • [BankAccountValidation.java](#) • [SecureBankAccount.java](#) • [BankingSystem.java](#)

OOP
Domain
Modeling

Time: $O(1)$
per
transaction

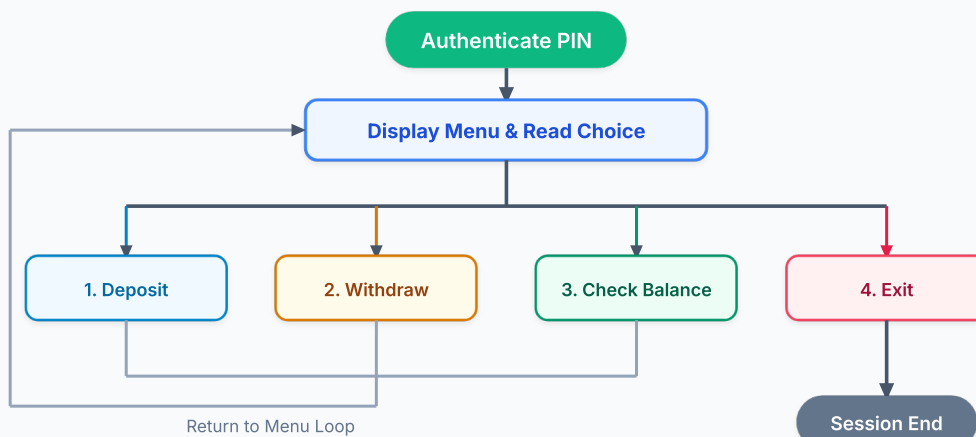
Space:
 $O(1)$

DETAILED DEFINITION & CORE CONCEPT

A canonical OOP domain modeling problem demonstrating business logic integrity. The account encapsulates `accountNumber`, `accountHolder`, and `balance`.

Business Invariants: - Deposits must be strictly positive ($\$amount > 0\$$). - Withdrawals must be positive ($\$amount > 0\$$) AND cannot exceed available funds ($\$amount \leq balance\$$).

ALGORITHMIC LOGIC & DECISION FLOWCHART



BankAccountClass.java

JAVA 17+

```
public class BankAccountClass {
    static class BankAccount {
        private String accNumber;
        private String holderName;
        private double balance;

        public BankAccount(String accNumber, String holderName, double initialBalance) {
            this.accNumber = accNumber;
            this.holderName = holderName;
            this.balance = Math.max(0, initialBalance);
        }

        public void deposit(double amount) {
            if (amount > 0) {
                balance += amount;
                System.out.println("Deposited $" + amount + ". New Balance: $" + balance);
            } else {
                System.out.println("Invalid deposit amount.");
            }
        }

        public void withdraw(double amount) {
            if (amount <= 0) {
                System.out.println("Withdrawal amount must be positive.");
            } else if (amount > balance) {
                System.out.println("Insufficient funds! Balance: $" + balance);
            }
        }
    }
}
```

```

        } else {
            balance -= amount;
            System.out.println("Withdrew $" + amount + ". Remaining Balance: $" + balance);
        }
    }

    public double getBalance() { return balance; }
}

public static void main(String[] args) {
    BankAccount acc = new BankAccount("ACC-9081", "John Doe", 500.0);
    acc.deposit(200.0);
    acc.withdraw(150.0);
    acc.withdraw(800.0); // Should be rejected
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize account with initial deposit.
2. Execute deposit method, validating \$amount > 0\$.
3. Execute withdrawal, verifying sufficient balance before deducting funds.
4. Reject overdraft attempts and preserve state integrity.

SAMPLE INPUT

Deposit 200 → Withdraw 150 → Withdraw 800

EXPECTED OUTPUT

Deposited \$200.0. New Balance: \$700.0\nWithdrew \$150.0. Remaining Balance: \$550.0\nInsufficient funds! Balance: \$550.0

Critical Edge Cases & Pitfalls

Overdraft Prevention: Attempting to withdraw \$800 with only \$550 balance correctly triggers rejection without modifying state.

Interactive ATM Console System

Interactive OOP Systems

Time: O(1) per operation

Space: O(1)

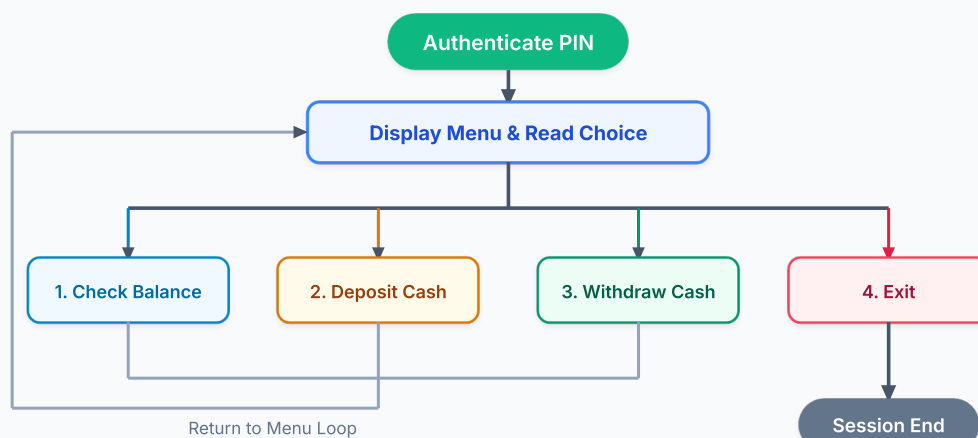
Files: ATM.java • ATMmenu.java • ATMSystem.java

DETAILED DEFINITION & CORE CONCEPT

An ATM terminal state machine coordinating user authentication, account balance inquiry, cash deposits, and withdrawals.

Separates view/menu logic from the underlying model (account business logic), providing an interactive menu loop that continues until the user explicitly logs out.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ATM.java

JAVA 17+

```
import java.util.Scanner;

public class ATMmenu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double balance = 1000.0;
        int choice;

        do {
            System.out.println("\n=== ATM MENU ===");
            System.out.println("1. Check Balance\n2. Deposit\n3. Withdraw\n4. Exit");
            System.out.print("Select choice (1-4): ");
            choice = sc.nextInt();

            switch (choice) {
                case 1 -> System.out.println("Current Balance: $" + balance);
                case 2 -> {
                    System.out.print("Enter deposit amount: ");
                    double dep = sc.nextDouble();
                    if (dep > 0) { balance += dep; System.out.println("Success! New Balance: $" + balance); }
                }
                case 3 -> {
                    System.out.print("Enter withdraw amount: ");
                    double withdraw = sc.nextDouble();
                    if (withdraw > 0) { balance -= withdraw; System.out.println("Success! New Balance: $" + balance); }
                }
                default -> System.out.println("Invalid choice.");
            }
        } while (choice != 4);
    }
}
```

```

        System.out.print("Enter withdrawal amount: ");
        double with = sc.nextDouble();
        if (with > 0 && with <= balance) { balance -= with; System.out.println("Cash Dispensed.
Balance: $" + balance); }
        else { System.out.println("Insufficient funds or invalid amount."); }
    }
    case 4 -> System.out.println("Thank you for banking with us. Goodbye!");
    default -> System.out.println("Invalid selection!");
}
} while (choice != 4);
sc.close();
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize starting balance.
2. Display 4-option CLI menu in a do-while loop.
3. Execute selected financial operation.
4. Loop until choice 4 (Exit) is entered.

SAMPLE INPUT

1 → 2 (Deposit 500) → 3 (Withdraw 200) → 4

EXPECTED OUTPUT

Current Balance: \$1000.0\nNew Balance:
\$1500.0\nCash Dispensed. Balance:
\$1300.0\nGoodbye!

Critical Edge Cases & Pitfalls

Zero/Negative Inputs: Intercepted before state mutation occurs, maintaining account ledger integrity.

MODULE 5 • CORE JAVA CURRICULUM

Inheritance, Polymorphism & Abstraction

Inheritance & Dynamic Method Overriding

Inheritance /
Polymorphism

Time:
O(1)

Space:
O(1)

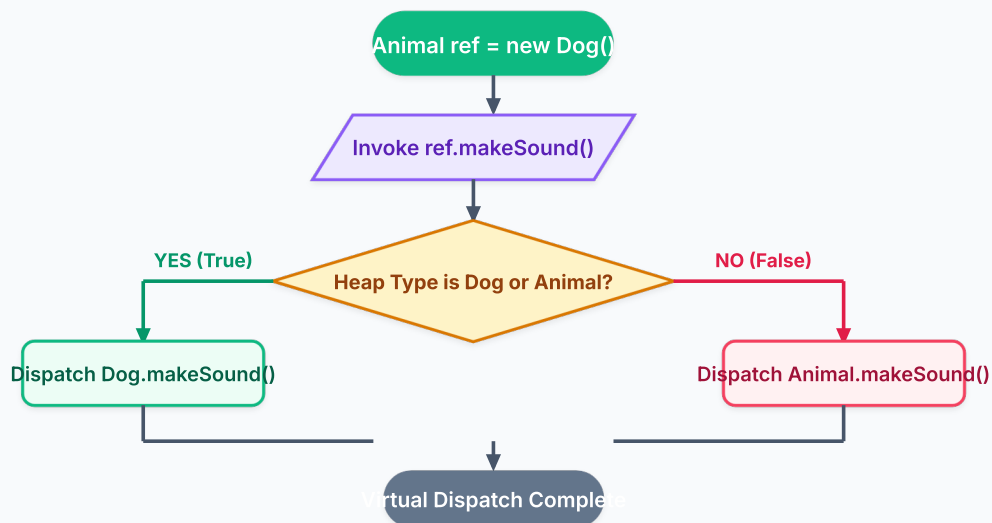
Files: [VehicleCar.java](#) • [AnimalDog.java](#) • [AnimalSounds.java](#) • [EmployeeHierarchy.java](#) • [MethodOverriding.java](#)

DETAILED DEFINITION & CORE CONCEPT

Inheritance establishes an IS-A relationship where a subclass inherits fields and methods from a superclass using `extends`.

Method Overriding occurs when a subclass provides a specific implementation of a method already declared in its superclass. At runtime, Java employs **Dynamic Method Dispatch (Virtual Method Invocation)**: the JVM determines which overridden version to invoke based on the actual object instance on the heap, NOT the variable reference type.

ALGORITHMIC LOGIC & DECISION FLOWCHART



VehicleCar.java

JAVA 17+

```
class Animal {
    public void makeSound() {
        System.out.println("Generic animal sound...");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Dog barks: Woof! Woof!");
    }
}

class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Cat meows: Meow! Meow!");
    }
}
```

```

    }
}

public class AnimalSounds {
    public static void main(String[] args) {
        // Polymorphic reference: Parent type pointing to Child objects
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.makeSound(); // Dynamically dispatched to Dog's method
        a2.makeSound(); // Dynamically dispatched to Cat's method
    }
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Define base class `Animal` with virtual method `makeSound()`.
2. Subclasses `Dog` and `Cat` override `makeSound()`.
3. Declare polymorphic references of type `Animal` assigned to subclass instances.
4. At runtime, JVM inspects the vtable and invokes the subclass implementation.

SAMPLE INPUT

Polymorphic calls: `a1.makeSound()`, `a2.makeSound()`

EXPECTED OUTPUT

Dog barks: Woof! Woof!\nCat meows: Meow! Meow!

Critical Edge Cases & Pitfalls

The @Override Annotation: Prevents silent bugs where an intentional override accidentally becomes an overload due to a parameter typo.

Abstract Classes & Template Design

Abstraction /
Polymorphism

Time:
O(1)

Space:
O(1)

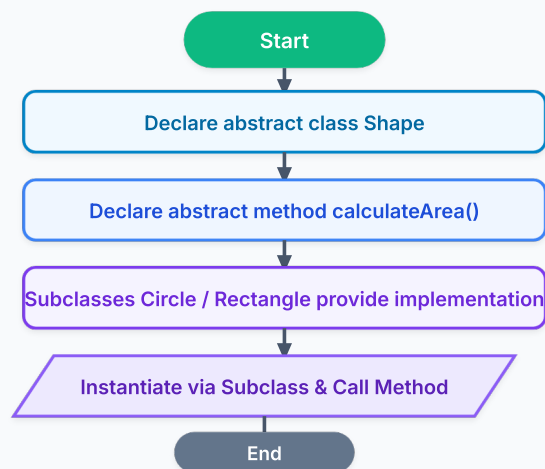
Files: AbstractShape.java • AbstractAnimal.java •
AbstractEmployee.java

DETAILED DEFINITION & CORE CONCEPT

An **abstract class** cannot be instantiated directly (using new) and serves as a common template for subclasses.

It can contain both: - **Abstract methods** (without a body), which subclasses *must* implement. - **Concrete methods** (with a full body) providing shared default behavior.

ALGORITHMIC LOGIC & DECISION FLOWCHART



AbstractShape.java

JAVA 17+

```
abstract class Shape {
    String color;
    public Shape(String color) { this.color = color; }

    // Abstract method: must be implemented by non-abstract children
    abstract double calculateArea();

    // Concrete method: shared common logic
    public void displayColor() {
        System.out.println("Shape Color: " + color);
    }
}

class Circle extends Shape {
    double radius;
    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }
    @Override
    double calculateArea() { return Math.PI * radius * radius; }
}

public class AbstractShape {
    public static void main(String[] args) {
        Shape c = new Circle("Crimson", 7.0);
        c.displayColor();
    }
}
```

```
        System.out.printf("Area: %.2f\n", c.calculateArea());
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Define abstract base class Shape with constructor and abstract method.
2. Subclass calls `super(color)` to initialize base fields.
3. Subclass provides concrete mathematical formula for `calculateArea()`.
4. Polymorphic execution via base class reference.

SAMPLE INPUT

Circle with color='Crimson', radius=7.0

EXPECTED OUTPUT

Shape Color: Crimson\nArea: 153.94

Critical Edge Cases & Pitfalls

Direct Instantiation Prohibition: `new Shape()` fails at compile time, enforcing design contracts.

Interfaces & Multiple Interface Implementation

Interfaces /
Contracts

Time:
O(1)

Space:
O(1)

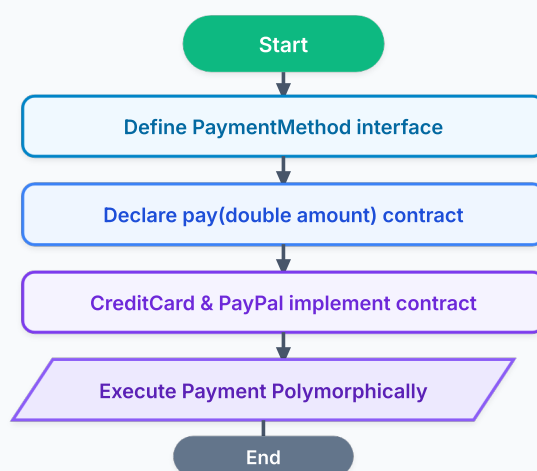
Files: CreatePaymentInterface.java • PaymentSystem.java • MultipleInterfaces.java • Shape.java

DETAILED DEFINITION & CORE CONCEPT

An **interface** is a formal contract specifying *what* an implementing class must do without dictating *how*.

Unlike classes, which only support single inheritance, a Java class can implement **multiple interfaces** (class C implements A, B), completely avoiding the C++ diamond problem because interfaces don't contain mutable instance state.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CreatePaymentInterface.java

JAVA 17+

```
interface PaymentMethod {
    void processPayment(double amount);
}

class CreditCardPayment implements PaymentMethod {
    private String cardNumber;
    public CreditCardPayment(String cardNumber) { this.cardNumber = cardNumber; }

    @Override
    public void processPayment(double amount) {
        System.out.println("Processing Credit Card payment of $" + amount + " for card: " + cardNumber);
    }
}

class UPIPayment implements PaymentMethod {
    private String upiId;
    public UPIPayment(String upiId) { this.upiId = upiId; }

    @Override
    public void processPayment(double amount) {
        System.out.println("Processing UPI payment of $" + amount + " to ID: " + upiId);
    }
}
```

```

public class PaymentSystem {
    public static void main(String[] args) {
        PaymentMethod p1 = new CreditCardPayment("4111-XXXX-XXXX-1234");
        PaymentMethod p2 = new UPIPayment("user@okbank");

        p1.processPayment(150.0);
        p2.processPayment(75.50);
    }
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Declare interface PaymentMethod with method signature processPayment.
2. Implement classes CreditCardPayment and UPIPayment.
3. Each class encapsulates its own provider attributes and logic.
4. Caller processes payments uniformly through the interface.

SAMPLE INPUT

CreditCard: 150.0, UPI: 75.50

EXPECTED OUTPUT

Processing Credit Card payment of \$150.0 for
card: 4111-XXXX-XXXX-1234\nProcessing UPI payment
of \$75.5 to ID: user@okbank

Critical Edge Cases & Pitfalls

Loose Coupling: New payment methods (e.g., Crypto, ApplePay) can be added without modifying existing checkout code.

MODULE 6 • CORE JAVA CURRICULUM

1D Array Algorithms: Searching, Extremes & Manipulations

Single-Pass Min, Max & Second Largest

Order
Statistics /
Arrays

Time:
 $O(N)$

Space:
 $O(1)$

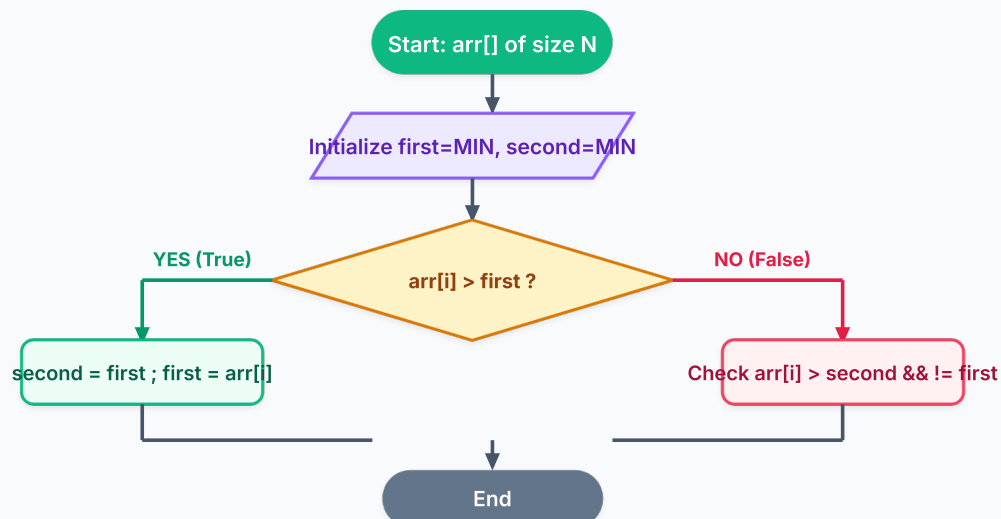
Files: [FindMinAndMax.java](#) • [FindtheLargestandSmallestNumber.java](#) • [FindtheSecondLargestNumber.java](#) • [SecondLargest.java](#)

DETAILED DEFINITION & CORE CONCEPT

Finding the second largest element in an unsorted array without sorting ($O(N \log N)$).

Using two tracker variables (`first` and `second`), we inspect each element x in a single pass: 1. If $x > \text{first}$, the former `first` cascades down to become `second`, and `first` takes x . 2. Else if $x > \text{second}$ and $x \neq \text{first}$, update `second = x`.

ALGORITHMIC LOGIC & DECISION FLOWCHART



FindMinAndMax.java

JAVA 17+

```
public class FindtheSecondLargestNumber {
    public static int findSecondLargest(int[] arr) {
        if (arr == null || arr.length < 2) return -1;

        int first = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > first) {
                second = first;
                first = num;
            } else if (num > second && num != first) {
                second = num;
            }
        }

        return (second == Integer.MIN_VALUE) ? -1 : second;
    }

    public static void main(String[] args) {
```

```
int[] arr = {12, 35, 1, 10, 34, 1};
System.out.println("Second Largest: " + findSecondLargest(arr));
}
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize first = Integer.MIN_VALUE and second = Integer.MIN_VALUE.
2. Iterate through each element num in the array.
3. If num > first, update second = first, then first = num.
4. Else if num > second and num != first (skip duplicate firsts), update second = num.
5. If second remained untouched, return -1 (no distinct second largest).

SAMPLE INPUT

Array: [12, 35, 1, 10, 34, 1]

EXPECTED OUTPUT

Second Largest: 34

Critical Edge Cases & Pitfalls

All Equal Elements: e.g., [7, 7, 7] correctly returns -1 because duplicate checks prevent second from being set to 7.

Two-Pointer In-Place Array Reversal

Two-Pointer / In-Place Swapping

Time: $O(N)$

Space: $O(1)$

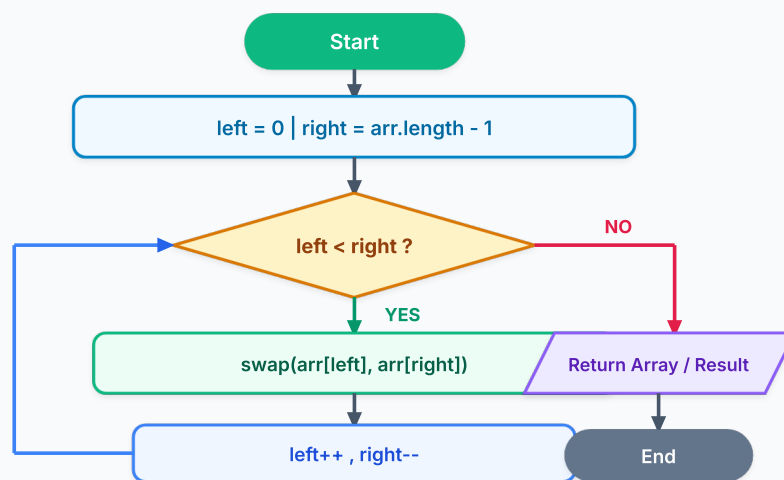
Files: ReverseanArray.java • ReverseArray.java

DETAILED DEFINITION & CORE CONCEPT

Reversing an array in-place without allocating a secondary buffer ($O(1)$ auxiliary space).

Two pointers converge toward each other: left initialized at 0, and right at $N - 1$. In each step, the values at left and right are swapped, then $\text{left}++$ and $\text{right}--$ until they meet or cross in the middle ($\lfloor N / 2 \rfloor$ swaps).

ALGORITHMIC LOGIC & DECISION FLOWCHART



ReverseanArray.java

JAVA 17+

```
import java.util.Arrays;

public class ReverseanArray {
    public static void reverse(int[] arr) {
        int left = 0;
        int right = arr.length - 1;

        while (left < right) {
            // In-place swap using temporary variable
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;

            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6};
        reverse(arr);
        System.out.println("Reversed: " + Arrays.toString(arr));
    }
}
```

```
}  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Set `left = 0` and `right = length - 1`.
2. Verify `left < right`.
3. Swap elements at indices `left` and `right`.
4. Increment `left`, decrement `right`.
5. Terminate when pointers cross.

SAMPLE INPUT

Array: [1, 2, 3, 4, 5, 6]

EXPECTED OUTPUT

Reversed: [6, 5, 4, 3, 2, 1]

Critical Edge Cases & Pitfalls

Odd vs Even Lengths: Even lengths swap all pairs. Odd lengths leave the central median element untouched without out-of-bounds errors.

Move Zeroes to End (Order Preserving)

Two-Pointer / In-Place
Partition

Time:
 $O(N)$

Space:
 $O(1)$

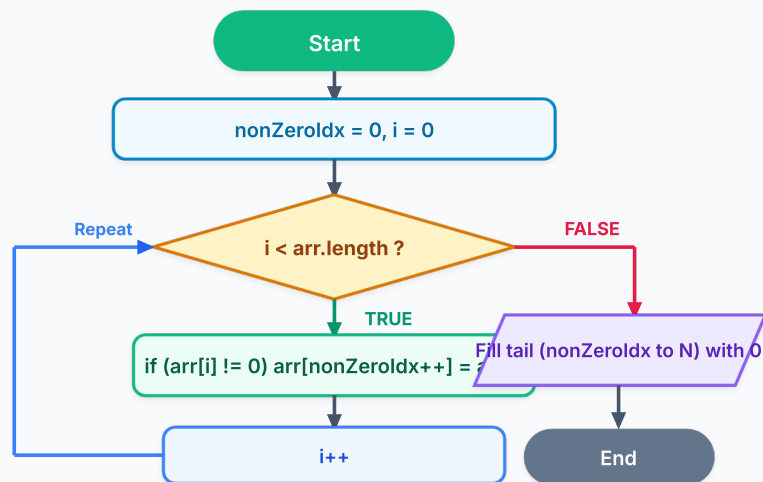
Files: MovesZeroesToEnd.java

DETAILED DEFINITION & CORE CONCEPT

Given an array of integers, move all 0s to the end of the array while maintaining the relative order of all non-zero elements.

Maintains an insertion index pointer `nonZeroIndex = 0`. As we iterate through the array, whenever a non-zero value is encountered, it is placed at `arr[nonZeroIndex++]`. After exhausting the array, the remainder of the array from `nonZeroIndex` to `N - 1` is filled with 0s.

ALGORITHMIC LOGIC & DECISION FLOWCHART



MovesZeroesToEnd.java

JAVA 17+

```
import java.util.Arrays;

public class MovesZeroesToEnd {
    public static void moveZeroes(int[] arr) {
        int nonZeroIdx = 0;

        // Pass 1: Compact non-zero elements to the front
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[nonZeroIdx++] = arr[i];
            }
        }

        // Pass 2: Fill trailing slots with zero
        while (nonZeroIdx < arr.length) {
            arr[nonZeroIdx++] = 0;
        }
    }

    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 3, 12};
        moveZeroes(arr);
    }
}
```

```
        System.out.println("Modified: " + Arrays.toString(arr));
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize write pointer nonZeroIdx = 0.
2. Traverse array: when arr[i] != 0, write to arr[nonZeroIdx] and increment nonZeroIdx.
3. After all non-zeros are placed, fill indices from nonZeroIdx to end with zeroes.

SAMPLE INPUT

Array: [0, 1, 0, 3, 12]

EXPECTED OUTPUT

Modified: [1, 3, 12, 0, 0]

Critical Edge Cases & Pitfalls

No Zeroes Present: The algorithm runs linearly without rewriting unnecessary values.

All Zeroes: Handled gracefully, remaining all zeroes.

Array Cyclic Rotation by K Offsets

Array Transformations

Time: $O(N)$

Space: $O(1)$

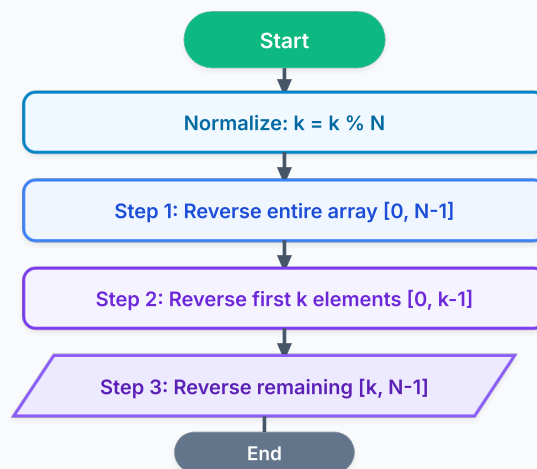
Files: RotateByArray.java

DETAILED DEFINITION & CORE CONCEPT

Rotating an array to the right by k positions moves the last k elements to the front.

The 3-Step Inversion Algorithm (Optimal $O(N)$ time, $O(1)$ space): 1. Normalize $k = k \% N$. 2. Reverse the entire array from index 0 to $N - 1$. 3. Reverse the first k elements from index 0 to $k - 1$. 4. Reverse the remaining $N - k$ elements from index k to $N - 1$.

ALGORITHMIC LOGIC & DECISION FLOWCHART



RotateByArray.java

JAVA 17+

```
import java.util.Arrays;

public class RotateByArray {
    public static void rotate(int[] arr, int k) {
        int n = arr.length;
        if (n <= 1) return;
        k = k % n;

        reverse(arr, 0, n - 1); // Reverse whole array
        reverse(arr, 0, k - 1); // Reverse first k
        reverse(arr, k, n - 1); // Reverse remaining
    }

    private static void reverse(int[] arr, int start, int end) {
        while (start < end) {
            int temp = arr[start];
            arr[start++] = arr[end];
            arr[end--] = temp;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7};
        rotate(arr, 3);
        System.out.println("Rotated by 3: " + Arrays.toString(arr));
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Compute $k = k \% n$ to handle rotations where $k \geq n$.
2. Perform 3 targeted reversals in place.
3. Array elements arrive at exact shifted positions.

SAMPLE INPUT

Array: [1, 2, 3, 4, 5, 6, 7], $k = 3$

EXPECTED OUTPUT

Rotated by 3: [5, 6, 7, 1, 2, 3, 4]

Critical Edge Cases & Pitfalls

$k = 0$ or $k = N$: Modulo yields 0, leaving the original array intact without redundant swaps.

Merge Two Sorted Arrays

Two-Pointer / Merging

Time: $O(N + M)$

Space: $O(N + M)$

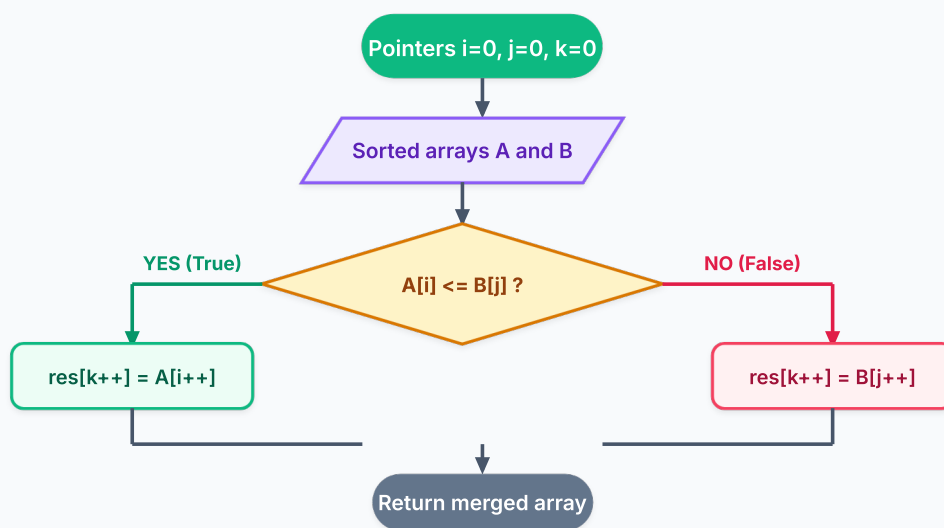
Files: `mergeTwoSortedArrays.java`

DETAILED DEFINITION & CORE CONCEPT

Combines two sorted arrays A of size N and B of size M into a single consolidated sorted array of size $N + M$.

Employs two pointers i and j indexing into A and B respectively. At each step, compare $A[i]$ with $B[j]$, append the smaller element to the result array, and advance the corresponding pointer. Once one array is exhausted, append the remaining elements of the other.

ALGORITHMIC LOGIC & DECISION FLOWCHART



`mergeTwoSortedArrays.java`

JAVA 17+

```
import java.util.Arrays;

public class mergeTwoSortedArrays {
    public static int[] merge(int[] a, int[] b) {
        int[] res = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;

        while (i < a.length && j < b.length) {
            if (a[i] <= b[j]) {
                res[k++] = a[i++];
            } else {
                res[k++] = b[j++];
            }
        }

        while (i < a.length) res[k++] = a[i++];
        while (j < b.length) res[k++] = b[j++];

        return res;
    }

    public static void main(String[] args) {
        int[] a = {1, 3, 5, 7};
        int[] b = {2, 4, 6, 8, 10};
    }
}
```

```
        System.out.println("Merged: " + Arrays.toString(merge(a, b)));  
    }  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Allocate target array of size `a.length + b.length`.
2. Compare elements pairwise while both pointers are within bounds.
3. Copy remaining elements from whichever array has leftover values.
4. Return complete merged array.

SAMPLE INPUT

`a = [1, 3, 5, 7], b = [2, 4, 6, 8, 10]`

EXPECTED OUTPUT

Merged: `[1, 2, 3, 4, 5, 6, 7, 8, 10]`

Critical Edge Cases & Pitfalls

Empty Array: If one array has length 0, the second array is copied directly.

MODULE 7 • CORE JAVA CURRICULUM

2D Arrays & Matrix Mathematics

Row-major multidimensional grids, nested coordinate iterations, matrix algebra (addition, dot product multiplication, transposition), diagonal traversal, and cell search.

Matrix Multiplication & Dimension Validation

2D Arrays / Linear Algebra

Time: $O(R_1 * C_1 * C_2)$

Space: $O(R_1 * C_2)$

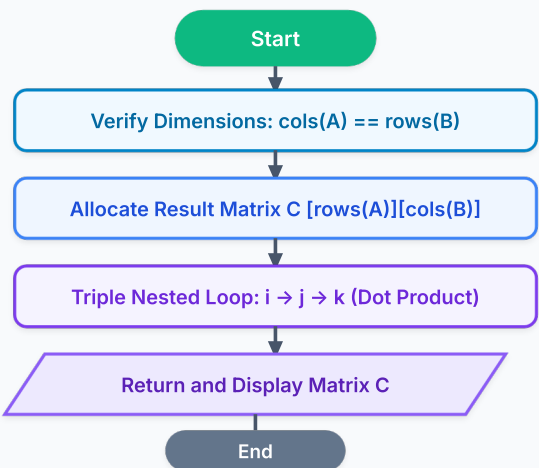
Files: [MatrixMultiplication.java](#) • [MatrixAddition.java](#)

DETAILED DEFINITION & CORE CONCEPT

The matrix product $C = A \times B$ is defined if and only if the number of columns in A equals the number of rows in B ($C_1 = R_2$).

Each entry $C[i][j]$ is the dot product of row i of matrix A with column j of matrix B : $C[i][j] = \sum_{k=0}^{C_1-1} A[i][k] \times B[k][j]$

ALGORITHMIC LOGIC & DECISION FLOWCHART



MatrixMultiplication.java

JAVA 17+

```
public class MatrixMultiplication {
    public static int[][] multiply(int[][] a, int[][] b) {
        int r1 = a.length, c1 = a[0].length;
        int r2 = b.length, c2 = b[0].length;

        if (c1 != r2) {
            System.out.println("Dimension mismatch! Cannot multiply.");
            return null;
        }

        int[][] res = new int[r1][c2];
        for (int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                for (int k = 0; k < c1; k++) {
                    res[i][j] += a[i][k] * b[k][j];
                }
            }
        }
        return res;
    }
}
```

```

public static void main(String[] args) {
    int[][] a = {{1, 2}, {3, 4}};
    int[][] b = {{2, 0}, {1, 2}};
    int[][] c = multiply(a, b);
    for (int[] row : c) {
        for (int val : row) System.out.print(val + " ");
        System.out.println();
    }
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Validate inner dimension equality: $c1 == r2$.
2. Allocate result array of dimensions $[r1][c2]$.
3. Outer loop i iterates rows of A, middle loop j iterates columns of B.
4. Inner loop k computes cumulative dot product sum.

SAMPLE INPUT

A = [[1,2],[3,4]], B = [[2,0],[1,2]]

EXPECTED OUTPUT

4 4\n10 8

Critical Edge Cases & Pitfalls

Incompatible Dimensions: e.g., 2×3 multiplied with 2×2 is caught by the guard, returning null safely.

Matrix Transposition

2D Coordinate Mapping

Time: $O(R * C)$

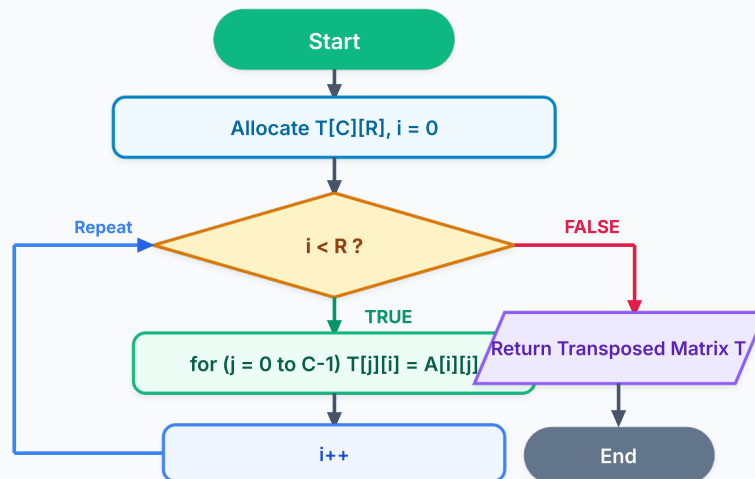
Space: $O(C * R)$

Files: TransposeofaMatrix.java

DETAILED DEFINITION & CORE CONCEPT

The transpose of a matrix A of dimensions $R \times C$ is a new matrix A^T of dimensions $C \times R$ obtained by flipping the matrix over its main diagonal: $A^T[j][i] = A[i][j]$. Rows of the original matrix become columns of the transposed matrix.

ALGORITHMIC LOGIC & DECISION FLOWCHART



TransposeofaMatrix.java

JAVA 17+

```
public class TransposeofaMatrix {
    public static int[][] transpose(int[][] matrix) {
        int r = matrix.length;
        int c = matrix[0].length;
        int[][] transposed = new int[c][r];

        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                transposed[j][i] = matrix[i][j];
            }
        }
        return transposed;
    }

    public static void main(String[] args) {
        int[][] mat = {{1, 2, 3}, {4, 5, 6}};
        int[][] t = transpose(mat);
        for (int[] row : t) {
            for (int val : row) System.out.print(val + " ");
            System.out.println();
        }
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Determine row count r and column count c .
2. Allocate new 2D array of size $[c][r]$.

3. Map coordinate values: `transposed[j][i] = matrix[i][j]`.

4. Display transposed grid.

SAMPLE INPUT

Matrix (2×3): `[[1, 2, 3], [4, 5, 6]]`

EXPECTED OUTPUT

`1 4\n2 5\n3 6`

Critical Edge Cases & Pitfalls

Non-Square (Rectangular) Matrices: Dimensions correctly swap from `2 × 3` to `3 × 2` without index bounds errors.

Main & Secondary Diagonal Sums

Square
Matrix
Analysis

Time: $O(N)$
single-loop
traversal

Space:
 $O(1)$

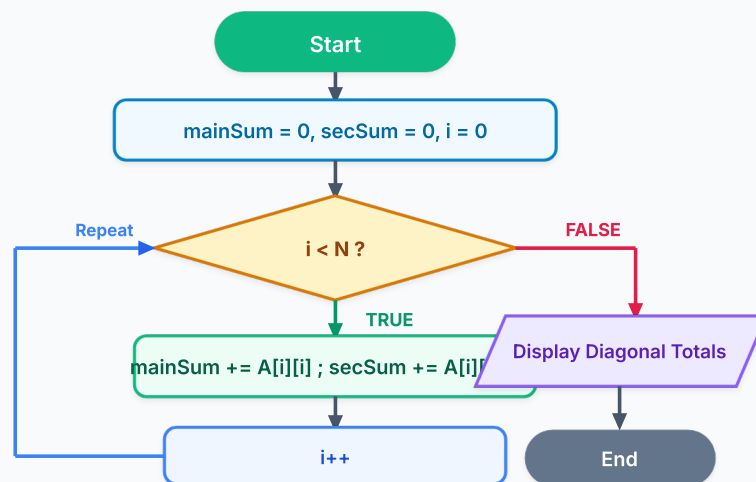
Files: [MainDiagonalSum.java](#) • [SecondaryDiagonalSum.java](#) • [FindtheSumofAllMatrixElements.java](#) • [SumofEachRow.java](#)

DETAILED DEFINITION & CORE CONCEPT

In an $N \times N$ square matrix: 1. **Main Diagonal:** Elements where row index equals column index ($i = j$). Element: $A[i][i]$. 2. **Secondary (Anti) Diagonal:** Elements where row index and column index sum to $N - 1$ ($i + j = N - 1$). Element: $A[i][N - 1 - i]$.

Both diagonals can be computed in a single $O(N)$ pass without a nested $O(N^2)$ scan.

ALGORITHMIC LOGIC & DECISION FLOWCHART



MainDiagonalSum.java

JAVA 17+

```
public class MainDiagonalSum {
    public static void computeDiagonals(int[][] matrix) {
        int n = matrix.length;
        int mainSum = 0;
        int secondarySum = 0;

        for (int i = 0; i < n; i++) {
            mainSum += matrix[i][i];
            secondarySum += matrix[i][n - 1 - i];
        }

        System.out.println("Main Diagonal Sum: " + mainSum);
        System.out.println("Secondary Diagonal Sum: " + secondarySum);
    }

    public static void main(String[] args) {
        int[][] mat = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        computeDiagonals(mat);
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize accumulators `mainSum = 0` and `secondarySum = 0`.
2. Run a single loop from `i = 0` to `N - 1`.
3. Add `matrix[i][i]` to main sum.
4. Add `matrix[i][n - 1 - i]` to secondary sum.
5. Output both diagonal sums.

SAMPLE INPUT

Matrix (3×3): `[[1,2,3],[4,5,6],[7,8,9]]`

EXPECTED OUTPUT

Main Diagonal Sum: 15\nSecondary Diagonal Sum: 15

Critical Edge Cases & Pitfalls

Center Element in Odd Dimensions: If calculating union of diagonals, the center element (`matrix[N/2][N/2]`) is shared and should only be counted once if deduplication is required.

MODULE 8 • CORE JAVA CURRICULUM

String Processing & Text Parsing Algorithms

String Palindrome Verification

Two-Pointer / String Processing

Time: $O(N)$

Space: $O(1)$

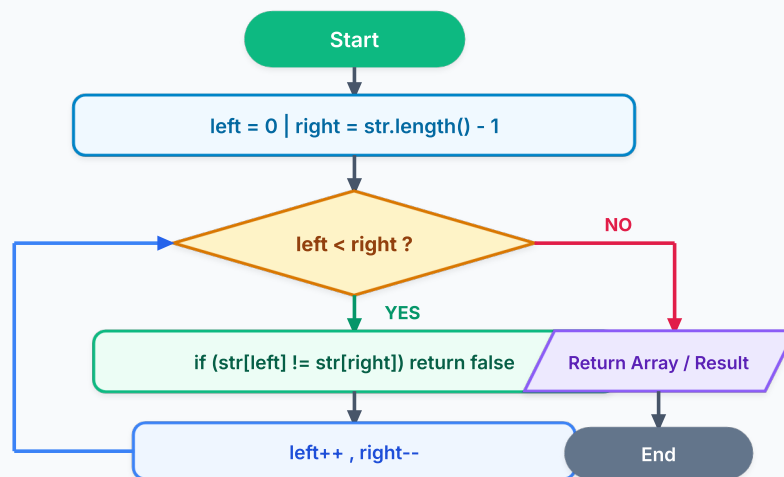
Files: [ReverseJava.java](#) • [ReversePalindrome.java](#) • [CheckPalindrome.java](#)

DETAILED DEFINITION & CORE CONCEPT

A string is a **palindrome** if it reads the exact same forward and backward (e.g., "radar", "madam").

Using two pointers `left = 0` and `right = s.length() - 1`, compare characters moving inward. If any mismatch occurs (`s.charAt(left) != s.charAt(right)`), immediately conclude false. Runs in $O(N)$ time with $O(1)$ auxiliary space.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ReverseJava.java

JAVA 17+

```
public class CheckPalindrome {
    public static boolean isPalindrome(String s) {
        if (s == null) return false;
        int left = 0;
        int right = s.length() - 1;

        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("'racecar' is palindrome? " + isPalindrome("racecar"));
        System.out.println("'hello' is palindrome? " + isPalindrome("hello"));
    }
}
```

```
}  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize pointers at string endpoints.
2. Compare characters at left and right positions.
3. If characters differ, early-exit and return false.
4. Increment left and decrement right.
5. If pointers meet without mismatches, return true.

SAMPLE INPUT

Inputs: 'racecar', 'hello'

EXPECTED OUTPUT

'racecar' is palindrome? true\n'hello' is
palindrome? false

Critical Edge Cases & Pitfalls

Single-Character & Empty Strings: Length 0 and length 1 strings are trivially palindromes (loop terminates immediately, returning true).

Valid Anagram Verification

Frequency Array /
Hashing

Time:
 $O(N)$

Space: $O(1)$ (fixed 26-element
array)

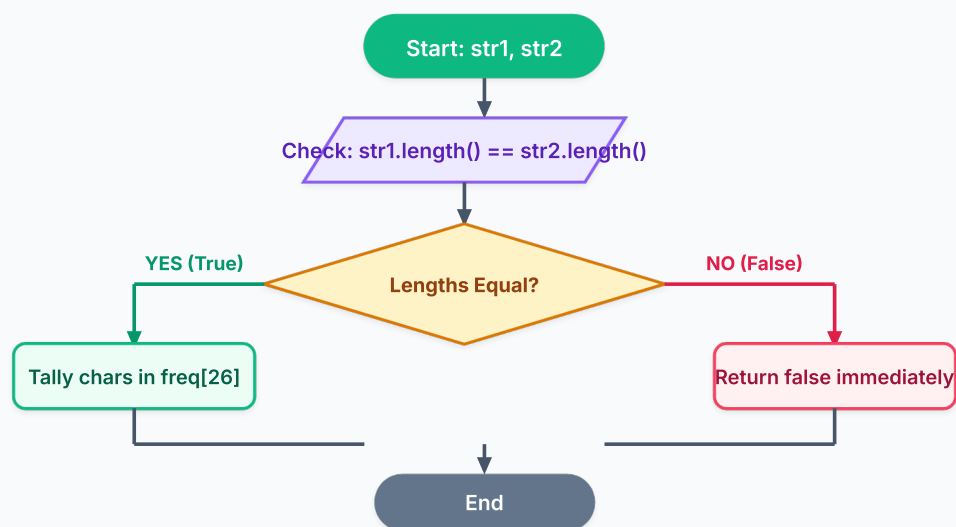
Files: ValidAnagram.java •
GroupAnagrams.java

DETAILED DEFINITION & CORE CONCEPT

Two strings are **anagrams** if they contain the exact same characters with identical frequencies, arranged in a different order (e.g., "silent" and "listen").

Optimized using a fixed-size frequency table of size 26 for ASCII lowercase characters: 1. If lengths differ, they cannot be anagrams. 2. Increment count for characters in string \$A\$. 3. Decrement count for characters in string \$B\$. 4. If all frequencies in the table equal zero, the strings are valid anagrams.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ValidAnagram.java

JAVA 17+

```
public class ValidAnagram {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;

        int[] freq = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            freq[s1.charAt(i) - 'a']++;
            freq[s2.charAt(i) - 'a']--;
        }

        for (int count : freq) {
            if (count != 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("anagram('listen', 'silent') -> " + isAnagram("listen", "silent"));
        System.out.println("anagram('rat', 'car') -> " + isAnagram("rat", "car"));
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Check length guard: if lengths differ, return false.
2. Initialize frequency buffer `int[26]`.
3. Increment tally for characters in `s1`, decrement for `s2` in one pass.
4. Scan buffer: if any non-zero count remains, return false; otherwise true.

SAMPLE INPUT

```
s1 = 'listen', s2 = 'silent'
```

EXPECTED OUTPUT

```
anagram('listen', 'silent') →  
true\nanagram('rat', 'car') → false
```

Critical Edge Cases & Pitfalls

Case Sensitivity: If uppercase letters are present, convert to uniform case with `toLowerCase()` or expand frequency table to 128/256 elements.

Longest Common Prefix (LCP)

Horizontal Scanning / Strings

Time: $O(S)$ where S is sum of all characters

Space: $O(1)$

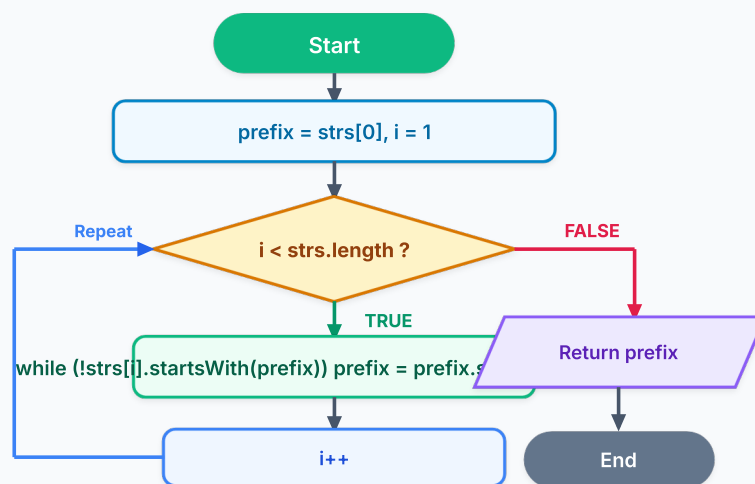
Files: LongestCommonPrefix.java

DETAILED DEFINITION & CORE CONCEPT

Finding the longest prefix substring shared across an array of strings (e.g., ["flower", "flow", "flight"] → "fl").

Horizontal Scanning Algorithm: Initialize `prefix = strs[0]`. Iteratively trim the prefix against each subsequent string `s` using `prefix = prefix.substring(0, prefix.length() - 1)` until `s.indexOf(prefix) == 0`. If prefix becomes empty, no common prefix exists.

ALGORITHMIC LOGIC & DECISION FLOWCHART



LongestCommonPrefix.java

JAVA 17+

```
public class LongestCommonPrefix {
    public static String longestCommonPrefix(String[] strs) {
        if (strs == null || strs.length == 0) return "";

        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }

    public static void main(String[] args) {
        String[] words = {"flower", "flow", "flight"};
        System.out.println("LCP: " + longestCommonPrefix(words));
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Set candidate prefix to the first word.

2. Iterate through subsequent words.
3. While the current word does not start with candidate prefix, truncate prefix by 1 character.
4. If prefix length reaches 0, return empty string.
5. Return remaining prefix.

SAMPLE INPUT

```
['flower', 'flow', 'flight']
```

EXPECTED OUTPUT

```
LCP: fl
```

Critical Edge Cases & Pitfalls

No Common Prefix: e.g., ["dog", "racecar", "car"] truncates prefix down to empty string "".

First Non-Repeating Character

Frequency Map / String Search

Time: $O(N)$

Space: $O(1)$ (fixed ASCII alphabet)

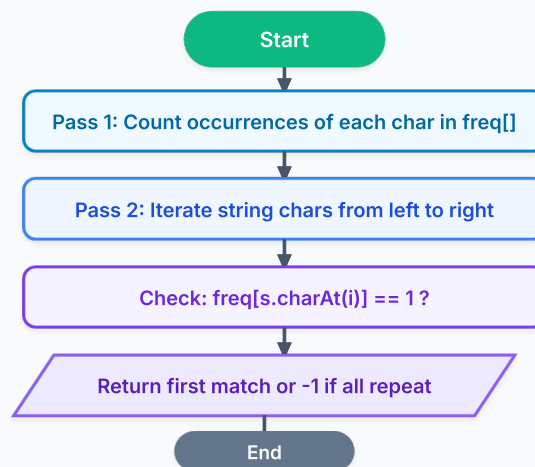
Files: FirstNonRepeating.java

DETAILED DEFINITION & CORE CONCEPT

Finding the first character in a string that appears exactly once.

Employs a two-pass algorithm: 1. **Pass 1**: Count character frequencies into a frequency table. 2. **Pass 2**: Scan string sequentially from left to right; return the first character whose tally in the frequency table equals 1.

ALGORITHMIC LOGIC & DECISION FLOWCHART



FirstNonRepeating.java

JAVA 17+

```
public class FirstNonRepeating {  
    public static char firstUnique(String s) {  
        int[] count = new int[256]; // Extended ASCII  
        for (char c : s.toCharArray()) count[c]++;  
  
        for (char c : s.toCharArray()) {  
            if (count[c] == 1) return c;  
        }  
        return '_'; // No unique char found  
    }  
  
    public static void main(String[] args) {  
        System.out.println("First unique in 'swiss': " + firstUnique("swiss"));  
        System.out.println("First unique in 'success': " + firstUnique("success"));  
    }  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Build character frequency table using first pass.
2. Traverse string in original order in second pass.
3. First character with count 1 is the result.
4. Return sentinel if no unique character exists.

SAMPLE INPUT

EXPECTED OUTPUT

'swiss'

First unique in 'swiss': w

Critical Edge Cases & Pitfalls

All Characters Repeating: e.g., "aabbcc" returns sentinel '_'.

MODULE 9 • CORE JAVA CURRICULUM

Recursion & Divide-and-Conquer

Recursive Factorial & Fibonacci Sequence

Recursion /
Divide-and-
Conquer

Time: Factorial: $O(N)$; Fib:
 $O(2^N)$ naive, $O(N)$ memoized

Space: $O(N)$
call stack
depth

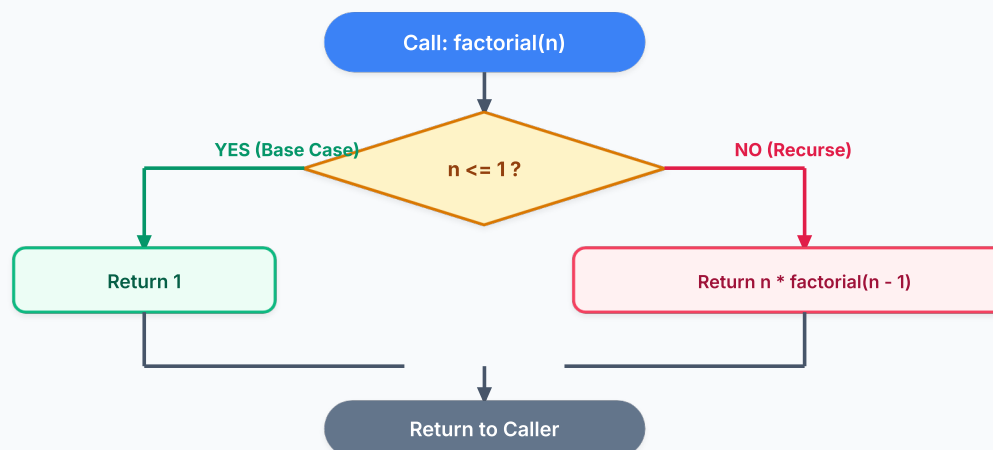
Files: [Factorial.java](#) • [Fibonacci.java](#) • [SumofDigitsusingRecursion.java](#)

DETAILED DEFINITION & CORE CONCEPT

A method that calls itself is **recursive**. Every recursive solution requires: 1. **Base Case**: A condition that terminates recursion without further self-invocation (prevents `StackOverflowError`). 2. **Recursive Step**: Reducing the problem instance toward the base case.

Recurrence relations: - Factorial: $F(n) = n \times F(n - 1)$ with $F(0) = 1$. - Fibonacci: $Fib(n) = Fib(n - 1) + Fib(n - 2)$ with $Fib(0) = 0$, $Fib(1) = 1$.

ALGORITHMIC LOGIC & DECISION FLOWCHART



Factorial.java

JAVA 17+

```
public class Factorial {
    public static long factorial(int n) {
        if (n <= 1) return 1; // Base case
        return n * factorial(n - 1); // Recursive step
    }

    public static long fibonacci(int n) {
        if (n <= 0) return 0;
        if (n == 1) return 1;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        System.out.println("Factorial(5) = " + factorial(5));
        System.out.println("Fibonacci(7) = " + fibonacci(7));
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Check base condition: if $n \leq 1$, return 1 immediately.
2. Otherwise, invoke `factorial(n - 1)` pushing frame onto stack.
3. Multiply return value by n while unwinding stack frames.

SAMPLE INPUT

`n = 5`

EXPECTED OUTPUT

`Factorial(5) = 120`
`Fibonacci(7) = 13`

Critical Edge Cases & Pitfalls

Deep Recursion: Recursive calls consume stack memory (~1MB default thread stack). For $n > 10,000$, recursion triggers `java.lang.StackOverflowError`.

Linked List Inversion via Recursive Call Stack

Recursion / Data Structures

Time: $O(N)$

Space: $O(N)$ call stack

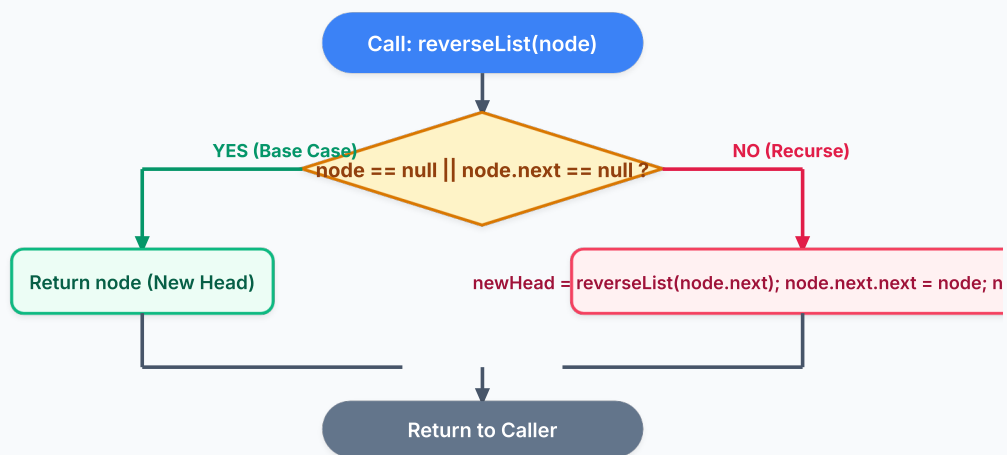
Files: ReverseLinkedListUsingRecursion.java

DETAILED DEFINITION & CORE CONCEPT

Reversing a singly linked list recursively relies on the JVM call stack to traverse to the end, identifying the new head, and reversing the pointers while unwinding.

For a node N , the recursive call reverses the tail. Upon return: $N.\text{next}.\text{next} = N$
 $N.\text{next} = \text{null}$ This makes the next node point backward to the current node, cleanly inverting the pointer direction.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ReverseLinkedListUsingRecursion.java

JAVA 17+

```
public class ReverseLinkedListUsingRecursion {
    static class Node {
        int data;
        Node next;
        Node(int data) { this.data = data; }
    }

    public static Node reverseList(Node current) {
        // Base case: empty list or reached terminal node
        if (current == null || current.next == null) {
            return current;
        }

        // Recursive call down to the last node
        Node newHead = reverseList(current.next);

        // Reverse connection: make successor point back to current
        current.next.next = current;
        current.next = null; // Sever old forward link

        return newHead;
    }

    public static void main(String[] args) {
```

```

Node head = new Node(1);
head.next = new Node(2);
head.next.next = new Node(3);

Node rev = reverseList(head);
while (rev != null) {
    System.out.print(rev.data + " -> ");
    rev = rev.next;
}
System.out.println("null");
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Recursively advance until reaching the final node (which becomes the new head).
2. At each stack level during unwinding, set `current.next.next = current`.
3. Sever existing forward link: `current.next = null`.
4. Propagate `newHead` back to the initial caller.

SAMPLE INPUT

1 → 2 → 3 → null

EXPECTED OUTPUT

3 → 2 → 1 → null

Critical Edge Cases & Pitfalls

Severing Link: Forgetting `current.next = null` creates a cyclic reference loop (1 → 2 → 1) causing infinite traversal loops.

Combinatorial Power Set Generation

Backtracking /
Combinatorics

Time:
 $O(2^N)$

Space: $O(N)$ recursion
tree depth

Files: `PrintAllSubsetsofAnArray.java`

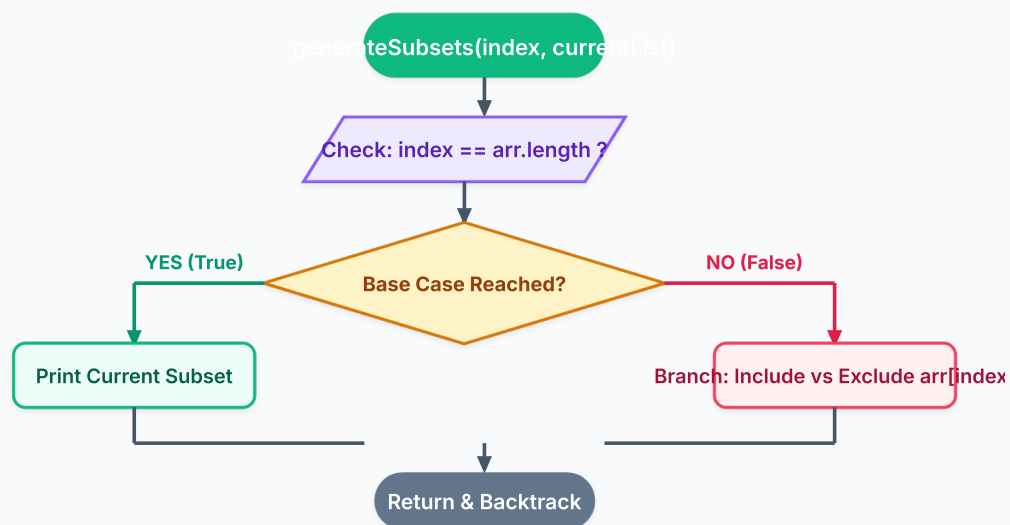
DETAILED DEFINITION & CORE CONCEPT

The power set of a set of size N contains all 2^N possible subsets (including the empty set and the set itself).

Using the **Include / Exclude Decision Tree**: At each index i , we branch into two recursive decisions: 1.

Include: Add element $arr[i]$ to the current subset and recurse to $i + 1$. 2. **Exclude**: Do not include $arr[i]$ and recurse to $i + 1$.

ALGORITHMIC LOGIC & DECISION FLOWCHART



`PrintAllSubsetsofAnArray.java`

JAVA 17+

```
import java.util.ArrayList;
import java.util.List;

public class PrintAllSubsetsofAnArray {
    public static void findSubsets(int[] arr, int index, List<Integer> current) {
        if (index == arr.length) {
            System.out.println(current);
            return;
        }

        // Choice 1: Include arr[index]
        current.add(arr[index]);
        findSubsets(arr, index + 1, current);

        // Backtrack
        current.remove(current.size() - 1);

        // Choice 2: Exclude arr[index]
        findSubsets(arr, index + 1, current);
    }

    public static void main(String[] args) {
```

```
int[] arr = {1, 2, 3};
findSubsets(arr, 0, new ArrayList<>());
}
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Base case: when `index == arr.length`, output current subset.
2. Add current element to list, recurse for next index (include branch).
3. Remove last element (backtrack) to restore state.
4. Recurse without adding current element (exclude branch).

SAMPLE INPUT

[1, 2, 3]

EXPECTED OUTPUT

[1, 2, 3]\n[1, 2]\n[1, 3]\n[1]\n[2, 3]\n[2]\n[3]\n[]

Critical Edge Cases & Pitfalls

Exponential Growth: A set of 20 elements generates $2^{20} = 1,048,576$ subsets. For $N > 25$, memory and CPU time grow rapidly.

MODULE 10 • CORE JAVA CURRICULUM

Advanced Algorithmic Paradigms (Two Pointers & Sliding Window)

Optimizing brute force $O(N^2)$ to linear $O(N)$ using two-pointer convergence, target difference hunting, and fixed/dynamic sliding window accumulators.

The Two Sum Problem

Files: [TwoSum.java](#) • [PairWithGivenSumInASortedArray.java](#)

Hashing /
Two
Pointers

Time: HashMap: $O(N)$ time, $O(N)$ space ; Sorted: $O(N)$ time, $O(1)$ space

Space: HashMap: $O(N)$; Two-Pointer: $O(1)$

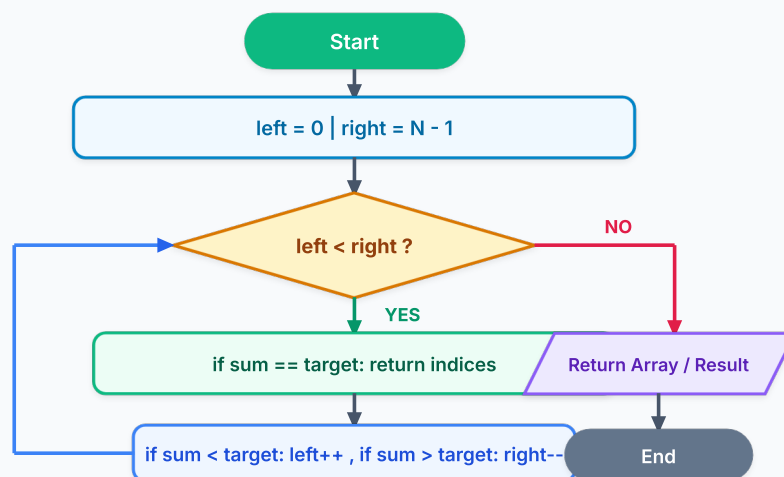
DETAILED DEFINITION & CORE CONCEPT

Finding two numbers in an array that sum to a given target value: $\text{arr}[i] + \text{arr}[j] = \text{target}$

1. Hash Map Approach ($O(N)$): For each element x , check if the complement $(\text{target} - x)$ already exists in the map.

2. Sorted Two-Pointer Approach ($O(N)$): Initialize $\text{left} = 0, \text{right} = N - 1$. - If $\text{sum} == \text{target}$: match found! - If $\text{sum} < \text{target}$: advance $\text{left}++$ (increase sum). - If $\text{sum} > \text{target}$: decrement $\text{right}--$ (decrease sum).

ALGORITHMIC LOGIC & DECISION FLOWCHART



TwoSum.java

JAVA 17+

```
import java.util.HashMap;
import java.util.Map;

public class TwoSum {
    // One-pass Hash Table Solution
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
            map.put(nums[i], i);
        }
        return new int[] {-1, -1};
    }
}
```

```
public static void main(String[] args) {  
    int[] nums = {2, 7, 11, 15};  
    int target = 9;  
    int[] res = twoSum(nums, target);  
    System.out.println("Indices: [" + res[0] + ", " + res[1] + "]");  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize hash map storing (value -> index).
2. Iterate through array calculating complement = target - nums[i].
3. If complement is in map, return indices immediately.
4. Otherwise, record current element and index in map.

SAMPLE INPUT

nums = [2, 7, 11, 15], target = 9

EXPECTED OUTPUT

Indices: [0, 1]

Critical Edge Cases & Pitfalls

Duplicate Elements: e.g., nums = [3, 3], target = 6. Checking for complement before inserting current element correctly finds index 0 when reading index 1.

Sliding Window: Maximum Sum Subarray of Size K

Sliding Window
Optimization

Time: $O(N)$
linear

Space:
 $O(1)$

Files: [MaximumSumSubarrayK.java](#) • [SubarrayWithGivenSum.java](#)

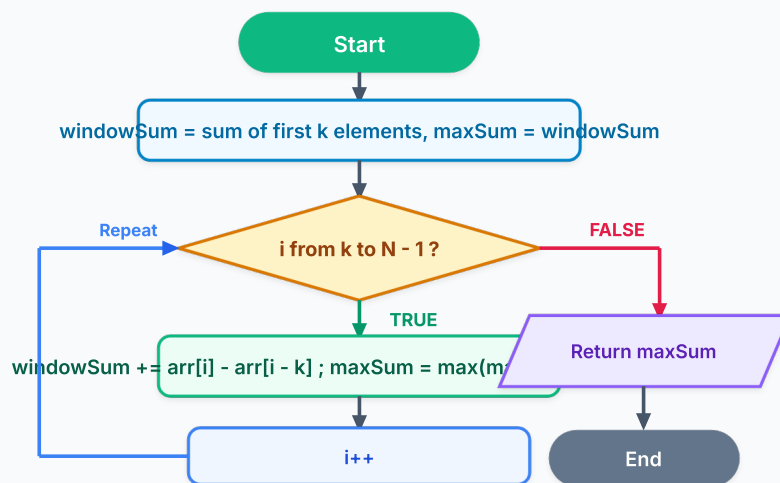
DETAILED DEFINITION & CORE CONCEPT

Finding the maximum sum of any contiguous subarray of fixed size k .

Brute force recalculates each window from scratch in $O(N \times k)$ time.

Sliding Window Optimization ($O(N)$): Compute the sum of the first window of size k . To slide the window one step to the right: $\text{newSum} = \text{oldSum} + \text{arr}[i] - \text{arr}[i - k]$ Add the incoming element and subtract the outgoing element in $O(1)$ constant time per step!

ALGORITHMIC LOGIC & DECISION FLOWCHART



MaximumSumSubarrayK.java

JAVA 17+

```
public class MaximumSumSubarrayK {
    public static int maxSubarraySum(int[] arr, int k) {
        if (arr.length < k) return -1;

        // Sum of initial window
        int windowSum = 0;
        for (int i = 0; i < k; i++) {
            windowSum += arr[i];
        }

        int maxSum = windowSum;
        // Slide the window across remainder of array
        for (int i = k; i < arr.length; i++) {
            windowSum += arr[i] - arr[i - k]; // Add incoming, drop outgoing
            maxSum = Math.max(maxSum, windowSum);
        }

        return maxSum;
    }
}
```

```
public static void main(String[] args) {  
    int[] arr = {2, 1, 5, 1, 3, 2};  
    int k = 3;  
    System.out.println("Max Sum Subarray of Size " + k + " = " + maxSubarraySum(arr, k));  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Compute sum of initial k elements.
2. Initialize $\text{maxSum} = \text{windowSum}$.
3. Slide window from index k to $N - 1$, adjusting by $+\text{arr}[i] - \text{arr}[i - k]$.
4. Update maxSum with the maximum value seen.
5. Return maxSum .

SAMPLE INPUT

`arr = [2, 1, 5, 1, 3, 2], k = 3`

EXPECTED OUTPUT

Max Sum Subarray of Size 3 = 9 (subarray [5, 1, 3])

Critical Edge Cases & Pitfalls

$k > N$: If window size exceeds array length, return -1 or throw `IllegalArgumentException`.

Pairs with Given Difference

Two-Pointer / Hashing

Time: $O(N)$ with HashSet or $O(N \log N)$ with Two-Pointer

Space: $O(N)$

Files:

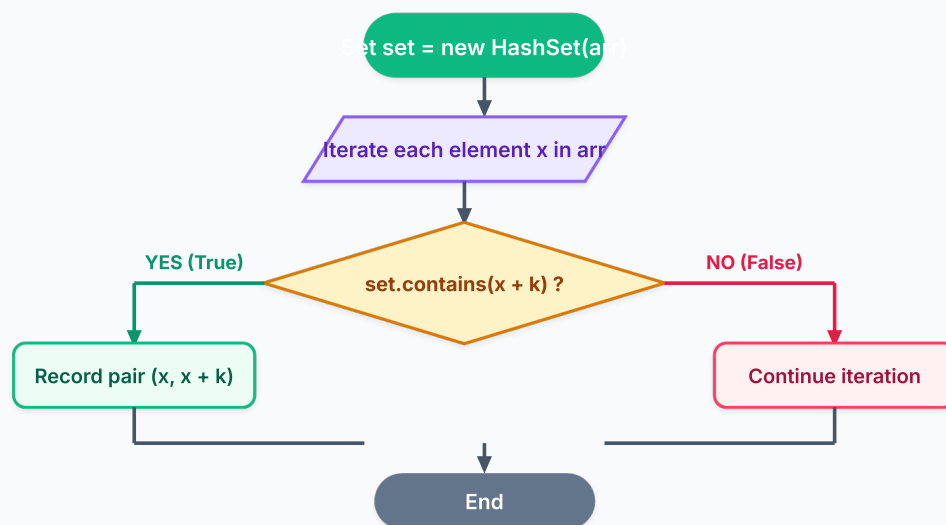
FindAllPairsWithAGivenDifference.java

DETAILED DEFINITION & CORE CONCEPT

Finding all distinct pairs (x, y) in an array such that their absolute difference equals a target value: $|x - y| = k$

Using a HashSet: insert all elements into the set. For each unique element x , check if $(x + k)$ exists in the set. If so, $(x, x + k)$ is a valid pair with difference k .

ALGORITHMIC LOGIC & DECISION FLOWCHART



FindAllPairsWithAGivenDifference.java

JAVA 17+

```
import java.util.HashSet;
import java.util.Set;

public class FindAllPairsWithAGivenDifference {
    public static void findPairs(int[] arr, int diff) {
        Set<Integer> set = new HashSet<>();
        for (int num : arr) set.add(num);

        System.out.println("Pairs with difference " + diff + ":");
        for (int num : set) {
            if (set.contains(num + diff)) {
                System.out.println("(" + num + ", " + (num + diff) + ")");
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 5, 3, 4, 2};
        int diff = 2;
        findPairs(arr, diff);
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Add array elements into a HashSet for $O(1)$ average-time lookups.
2. Iterate through unique numbers in the set.
3. Check if $\text{num} + \text{diff}$ is present.
4. Print matching pairs.

SAMPLE INPUT

`arr = [1, 5, 3, 4, 2], diff = 2`

EXPECTED OUTPUT

`Pairs with difference 2:\n(1, 3)\n(2, 4)\n(3, 5)`

Critical Edge Cases & Pitfalls

Difference $k = 0$: Requires checking for duplicate elements in the original array rather than looking up $\text{num} + 0$.

MODULE 11 • CORE JAVA CURRICULUM

Geometric CLI Pattern Printing

Nested loop coordinate geometry, row-column mapping, leading whitespace indentation formulas, and CLI shape rendering.

Geometric CLI Pattern Printing

Nested Loops /
Coordinate
Geometry

Time:
 $O(N^2)$

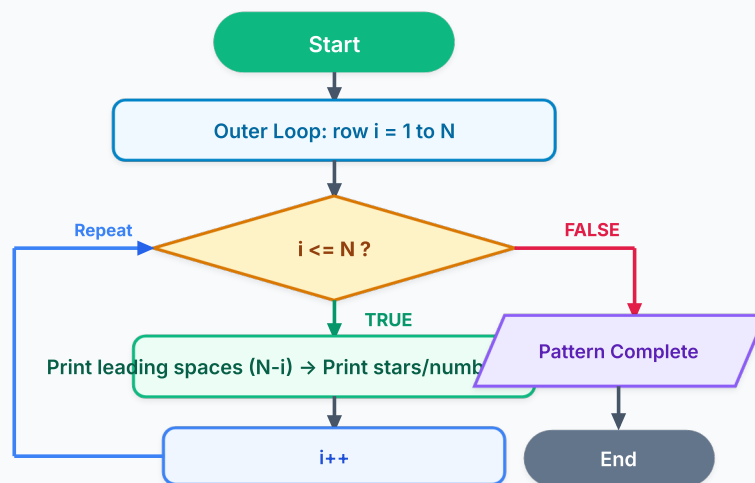
Space:
 $O(1)$

Files: [SquarePattern.java](#) • [IncreasingTriangle.java](#) •
[NumberTriangle.java](#) • [ReverseTriangle.java](#) • [NumberPyramid.java](#)

DETAILED DEFINITION & CORE CONCEPT

CLI geometric patterns build mental models for nested coordinate systems: - **Outer Loop (i)**: Governs the vertical row axis ($1 \leq i \leq N$). - **Inner Spaces Loop (s)**: Formulates horizontal indentation ($N - i$ leading spaces for centered pyramids). - **Inner Content Loop (j)**: Prints symbols or numbers on the active row ($1 \leq j \leq 2i - 1$ for pyramids).

ALGORITHMIC LOGIC & DECISION FLOWCHART



SquarePattern.java

JAVA 17+

```
public class NumberPyramid {
    public static void printPyramid(int n) {
        for (int i = 1; i <= n; i++) {
            // Print leading spaces for alignment
            for (int s = 1; s <= n - i; s++) {
                System.out.print(" ");
            }
            // Print ascending numbers
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        printPyramid(5);
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

- 1. Outer loop runs from row 1 up to \$N\$.
- 2. Inner loop prints \$(N - i)\$ spaces to center the current row.
- 3. Second inner loop prints numbers \$1\$ to \$i\$ separated by spaces.
- 4. Print newline character at the end of each row.

SAMPLE INPUT

n = 5

EXPECTED OUTPUT

1 \n 1 2 \n 1 2 3 \n 1 2 3 4 \n1 2 3 4 5

Critical Edge Cases & Pitfalls

n = 1: Outputs a single centered value without misplaced leading or trailing spaces.

MODULE 12 • CORE JAVA CURRICULUM

Exception Handling & Robust System Architecture

Exception Handling Mechanics (try-catch-finally)

Runtime
Safety /
Exceptions

Time:
O(1)
regular
flow

Space:
O(1)

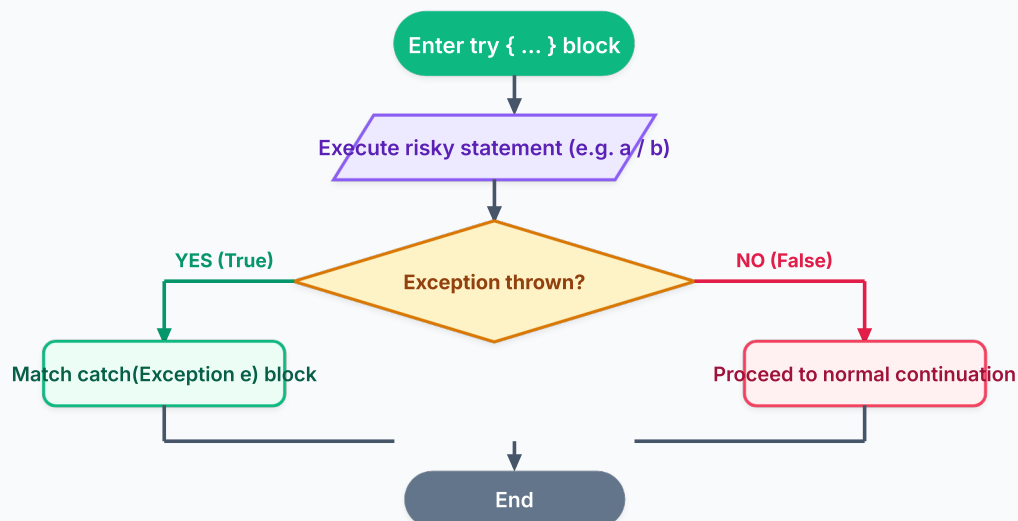
Files: [DivisionException.java](#) • [ArrayException.java](#) •
[MultipleExceptions.java](#) • [CustomException.java](#) •
[BankAccountValidationException.java](#)

DETAILED DEFINITION & CORE CONCEPT

Java divides exceptions into two main branches under `java.lang.Throwable`: 1. **Checked Exceptions**: Subclasses of `Exception` (excluding `RuntimeException`). Must be either caught in a try-catch or declared in a throws clause (e.g. `IOException`, `SQLException`). 2. **Unchecked Exceptions**: Subclasses of `RuntimeException`. Indicate programming or algorithmic logic bugs (e.g. `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `NullPointerException`).

The `finally` block is guaranteed to execute regardless of whether an exception is thrown or caught, making it the standard location for resource cleanup.

ALGORITHMIC LOGIC & DECISION FLOWCHART



DivisionException.java

JAVA 17+

```
public class DivisionException {
    public static void divide(int a, int b) {
        try {
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.err.println("Caught Exception: Division by zero is undefined! " + e.getMessage());
        } finally {
            System.out.println("Cleanup: Execution completed successfully.");
        }
    }

    public static void main(String[] args) {
```

```
        divide(10, 2);  
        divide(10, 0); // Gracefully caught without process crash  
    }  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Encapsulate risky operations inside a try block.
2. If an arithmetic error occurs, runtime creates an `ArithmeticException` object.
3. Execution jumps directly to the matching catch block, preventing JVM termination.
4. The finally block executes in all scenarios.

SAMPLE INPUT

`divide(10, 0)`

EXPECTED OUTPUT

Caught Exception: Division by zero is undefined!
/ by zero\nCleanup: Execution completed
successfully.

Critical Edge Cases & Pitfalls

Catch Order: Subclass exceptions (e.g. `ArithmeticException`) must precede superclass exceptions (e.g. `Exception`); otherwise the code will fail to compile with an 'unreachable catch block' error.

Boss Challenge: Enterprise ATM Exception System

Custom Exception
Architecture

Time: O(1) per
operation

Space:
O(1)

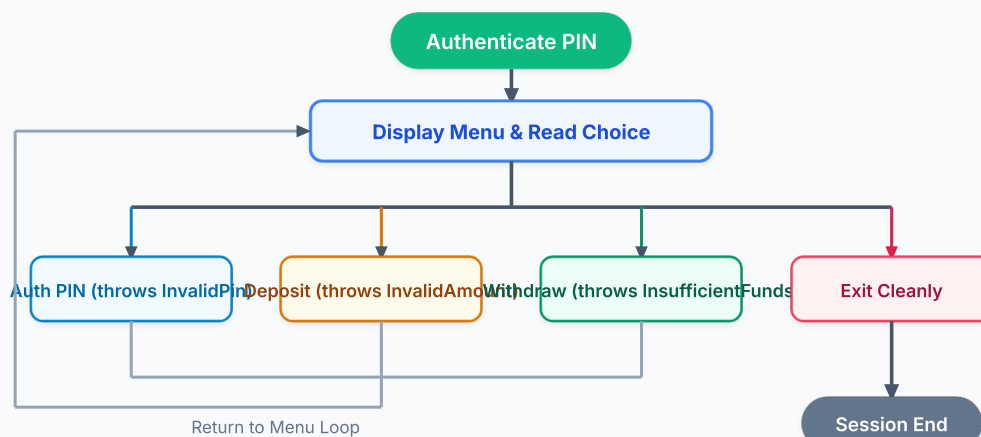
Files: `ATMExceptionSystem.java`

DETAILED DEFINITION & CORE CONCEPT

Domain-driven exception design replaces generic error messages with meaningful, strongly-typed custom exceptions: - `InvalidPinException` (Unsuccessful authentication) - `InsufficientBalanceException` (Overdraft prevention) - `InvalidAmountException` (Zero or negative financial operations)

Each custom exception encapsulates error codes and contextual messages, allowing calling layers to handle specific failure modes individually.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ATMExceptionSystem.java

JAVA 17+

```
// Custom Domain Exceptions
class InvalidPinException extends Exception {
    public InvalidPinException(String msg) { super(msg); }
}

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) { super(msg); }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String msg) { super(msg); }
}

public class ATMExceptionSystem {
    private static double balance = 2500.0;
    private static final String CORRECT_PIN = "4321";

    public static void authenticate(String pin) throws InvalidPinException {
        if (!CORRECT_PIN.equals(pin)) {
            throw new InvalidPinException("Security Alert: Invalid PIN entered!");
        }
    }

    public static void withdraw(double amount) throws InsufficientBalanceException, InvalidAmountException
```

```

{
    if (amount <= 0) throw new InvalidAmountException("Withdrawal amount must be > $0");
    if (amount > balance) throw new InsufficientBalanceException("Insufficient funds! Available: $" +
balance);
    balance -= amount;
    System.out.println("Withdrew $" + amount + ". Remaining Balance: $" + balance);
}

public static void main(String[] args) {
    try {
        authenticate("4321");
        withdraw(3000.0); // Triggers custom exception
    } catch (InvalidPinException | InsufficientBalanceException | InvalidAmountException e) {
        System.err.println("Transaction Failed: " + e.getMessage());
    }
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Declare custom checked exceptions extending `java.lang.Exception`.
2. Methods declare potential exceptions using the `throws` keyword.
3. Business logic triggers errors explicitly using `throw new ...`.
4. Caller intercepts domain exceptions using multi-catch blocks.

SAMPLE INPUT

PIN='4321', withdraw(3000.0)

EXPECTED OUTPUT

Transaction Failed: Insufficient funds!
Available: \$2500.0

Critical Edge Cases & Pitfalls

State Invariance: Throwing the exception before mutating balance ensures the account is never left in an inconsistent state.

MODULE 13 • CORE JAVA CURRICULUM

Collections Framework (List, Set, Map)

Collections Framework: List, Set, and Map

Files: ArrayListStudentNames.java • ArrayListNumbers.java • RemoveDuplicatesHashSet.java • IterateThroughHashSet.java • ListVsSet.java • StudentMarksHashMap.java • WordFrequencyHashMap.java • EmployeeMap.java

Collections
Core

Time: ArrayList:
O(1) get, O(N)
insert ;
HashSet/HashMap:
O(1) average
lookup

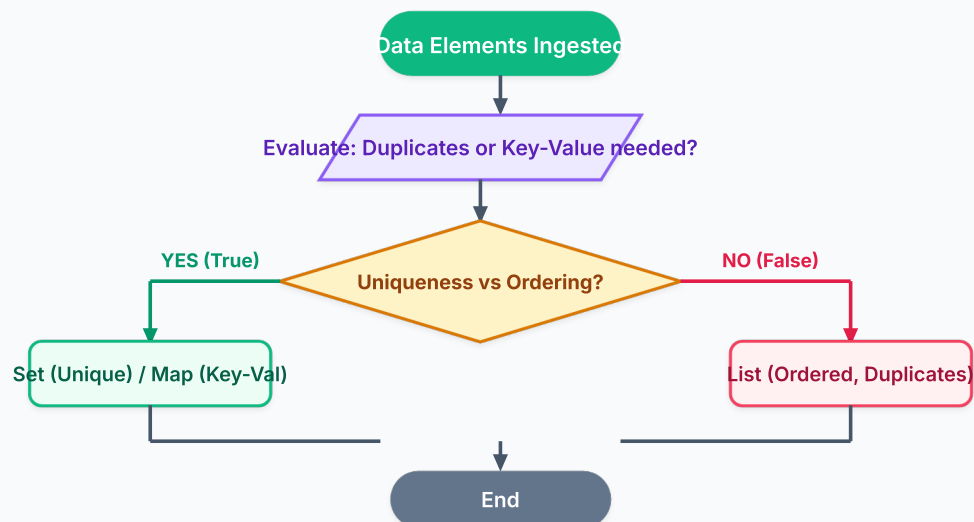
Space:
O(N)

DETAILED DEFINITION & CORE CONCEPT

The Java Collections Framework provides three primary interfaces:

1. **List (ArrayList):** An ordered, index-accessible sequence that allows duplicate elements. Backed by a dynamically resizing array with $1.5\times$ growth factor.
2. **Set (HashSet):** An unordered collection of unique elements that rejects duplicates. Backed internally by a HashMap; relies on hashCode() and equals().
3. **Map (HashMap):** Associative key-value store (O(1) average get/put). Uses hash buckets with balanced red-black trees for buckets with ≥ 8 colliding elements.

ALGORITHMIC LOGIC & DECISION FLOWCHART



ArrayListStudentNames.java

JAVA 17+

```
import java.util.*;

public class ListVsSet {
    public static void main(String[] args) {
        Integer[] rawData = {10, 20, 10, 30, 20, 40};

        // List preserves insertion order and allows duplicates
        List<Integer> list = new ArrayList<>(Arrays.asList(rawData));
        System.out.println("List (allows duplicates): " + list);
        System.out.println("List Size: " + list.size());

        // Set enforces uniqueness, automatically discarding duplicates
```

```

Set<Integer> set = new LinkedHashSet<>(Arrays.asList(rawData));
System.out.println("Set (unique, preserved order): " + set);
System.out.println("Set Size: " + set.size());

// Map stores key-value pairs
Map<String, Integer> map = new HashMap<>();
map.put("Alice", 95);
map.put("Bob", 82);
System.out.println("Map Lookup Alice: " + map.get("Alice"));
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Initialize an ArrayList with duplicate elements and verify all are retained.
2. Pass the same elements to a Set; duplicates are automatically filtered out via hash code evaluation.
3. Store key-value associations in a HashMap and retrieve values in $O(1)$ time.

SAMPLE INPUT

```
rawData = [10, 20, 10, 30, 20, 40]
```

EXPECTED OUTPUT

```
List: [10, 20, 10, 30, 20, 40] (Size: 6)\nSet:
[10, 20, 30, 40] (Size: 4)\nMap Lookup Alice: 95
```

Critical Edge Cases & Pitfalls

Custom Objects in Sets/Maps: Storing custom objects requires properly overriding both `equals()` and `hashCode()` to prevent duplicate entries.

Boss Challenge: Student Management System

Integrated OOP / Collections

Time: $O(1)$ lookup via Map, $O(N)$ list traversal

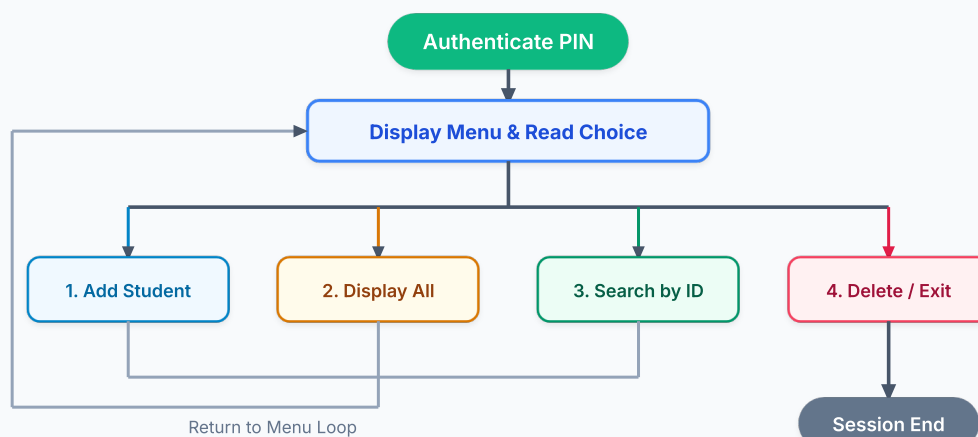
Space: $O(N)$

Files: StudentManagementSystem.java • StudentManagement.java

DETAILED DEFINITION & CORE CONCEPT

An integrated CRUD system demonstrating enterprise collections architecture: - `ArrayList<Student>` maintains ordered student records. - `HashMap<Integer, Student>` enables instantaneous $O(1)$ lookups by student ID. - Custom domain exceptions (`StudentNotFoundException`, `DuplicateStudentException`) enforce business rules.

ALGORITHMIC LOGIC & DECISION FLOWCHART



StudentManagementSystem.java

JAVA 17+

```
import java.util.*;

class Student {
    private int id;
    private String name;
    private double marks;

    public Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getMarks() { return marks; }
    @Override
    public String toString() {
        return "Student[ID=" + id + ", Name=" + name + ", Marks=" + marks + "];"
    }
}

public class StudentManagementSystem {
    private Map<Integer, Student> studentMap = new HashMap<>();

    public void addStudent(Student s) {
```

```

        if (studentMap.containsKey(s.getId())) {
            System.out.println("Error: Student ID " + s.getId() + " already exists!");
            return;
        }
        studentMap.put(s.getId(), s);
        System.out.println("Student added successfully: " + s.getName());
    }

    public void searchStudent(int id) {
        Student s = studentMap.get(id);
        if (s != null) {
            System.out.println("Found: " + s);
        } else {
            System.out.println("Student with ID " + id + " not found!");
        }
    }

    public static void main(String[] args) {
        StudentManagementSystem sms = new StudentManagementSystem();
        sms.addStudent(new Student(101, "David", 88.5));
        sms.addStudent(new Student(102, "Emma", 94.0));
        sms.searchStudent(101);
        sms.searchStudent(999); // Not found test
    }
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Encapsulate student fields within a Student class.
2. Coordinate primary storage using HashMap.
3. Check for duplicates with `containsKey()` before inserting.
4. Retrieve records by ID in $O(1)$ time.

SAMPLE INPUT

Add ID 101, Add ID 102, Search ID 101, Search ID 999

EXPECTED OUTPUT

Student added: David\nStudent added: Emma\nFound: Student[ID=101, Name=David, Marks=88.5]\nStudent with ID 999 not found!

Critical Edge Cases & Pitfalls

Duplicate IDs: Intercepted before inserting, preventing existing student records from being silently overwritten.

MODULE 14 • CORE JAVA CURRICULUM

Object Sorting, Comparable & Comparator

Natural vs Custom Sorting: Comparable vs Comparator

Sorting /
Functional
Interfaces

Time: $O(N \log N)$ via
TimSort

Space:
 $O(N)$
auxiliary
space

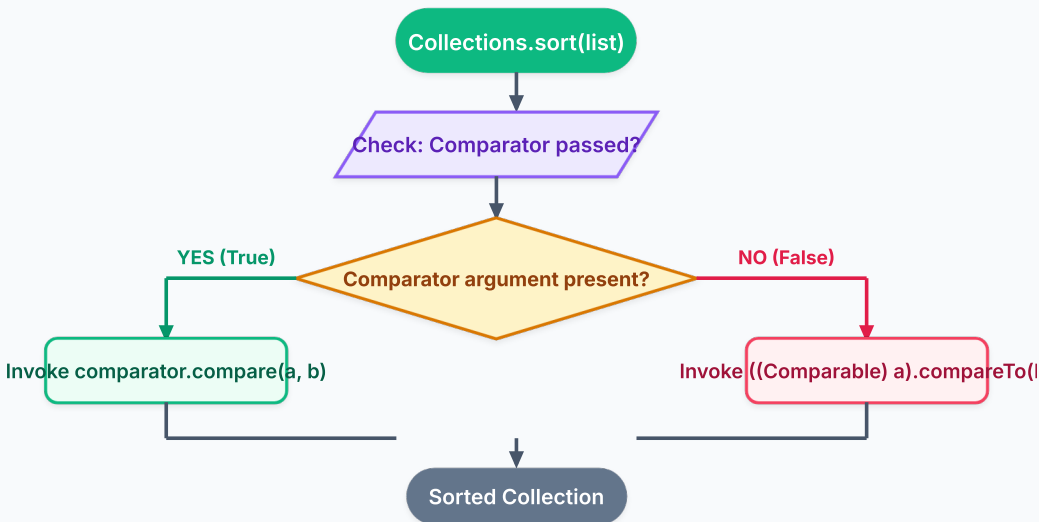
Files: SortNumbers.java • SortStudentNames.java •
StudentSortingWithComparable.java •
StudentSortingWithComparator.java • EmployeeSortingSystem.java

DETAILED DEFINITION & CORE CONCEPT

Java provides two distinct interfaces for ordering objects:

- Comparable<T> (Natural Ordering):** - Implemented directly by the domain class itself via `int compareTo(T o)`. - Located in package `java.lang`. - Defines a single default sort order (e.g. by Student ID or Marks).
- Comparator<T> (Custom Strategy Ordering):** - Implemented in external classes or lambda expressions via `int compare(T o1, T o2)`. - Located in package `java.util`. - Allows multiple independent sorting strategies on the same class (e.g. by Name, by Salary, or by Age).

ALGORITHMIC LOGIC & DECISION FLOWCHART



SortNumbers.java

JAVA 17+

```
import java.util.*;

class Student implements Comparable<Student> {
    int id;
    String name;
    double marks;

    public Student(int id, String name, double marks) {
        this.id = id; this.name = name; this.marks = marks;
    }

    // Natural ordering: Ascending by marks
    @Override
```

```

    public int compareTo(Student other) {
        return Double.compare(this.marks, other.marks);
    }

    @Override
    public String toString() { return name + " (" + marks + ")"; }
}

public class StudentSortingwithComparator {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student(1, "Charlie", 76.5));
        list.add(new Student(2, "Alice", 92.0));
        list.add(new Student(3, "Bob", 85.0));

        // 1. Natural Sort via Comparable (by marks ascending)
        Collections.sort(list);
        System.out.println("By Marks (Comparable): " + list);

        // 2. Custom Strategy via Comparator (by name alphabetically)
        list.sort((s1, s2) -> s1.name.compareTo(s2.name));
        System.out.println("By Name (Comparator): " + list);
    }
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Implement Comparable inside class definition for default ordering.
2. Invoke Collections.sort(list) using compareTo().
3. Supply a custom lambda Comparator to sort by name alphabetically.
4. Observe how both sorting strategies order the list correctly.

SAMPLE INPUT

[Charlie: 76.5, Alice: 92.0, Bob: 85.0]

EXPECTED OUTPUT

By Marks: [Charlie (76.5), Bob (85.0), Alice (92.0)]\nBy Name: [Alice (92.0), Bob (85.0), Charlie (76.5)]

Critical Edge Cases & Pitfalls

Floating-Point Subtraction in compare: Never use (int)(this.marks - other.marks) because fractional differences (e.g. 85.4 vs 85.8) truncate to 0. Always use Double.compare(a, b).

Boss Challenge: Employee Multi-Criteria Sorting

Complex Sorting Systems

Time: $O(N \log N)$

Space: $O(N)$

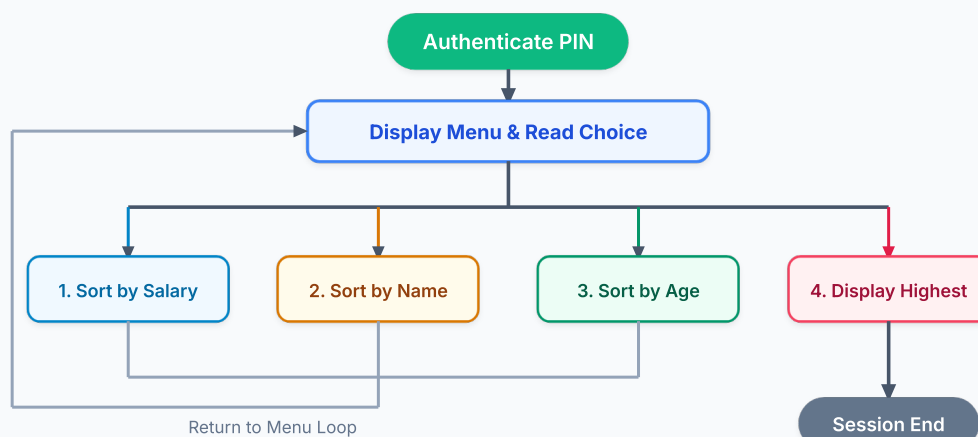
Files: `EmployeeSortingSystem.java`

DETAILED DEFINITION & CORE CONCEPT

An enterprise employee ranking system providing multiple sorting strategies on demand: 1. Sort by Salary (Descending) 2. Sort by Name (Alphabetical) 3. Sort by Age (Ascending) 4. Compound Sorting (e.g. by Department, then by Salary).

Implements clean separation of concerns using `Comparator.comparing()` and `thenComparing()` chains.

ALGORITHMIC LOGIC & DECISION FLOWCHART



EmployeeSortingSystem.java

JAVA 17+

```
import java.util.*;

class Employee {
    int id;
    String name;
    double salary;
    int age;

    public Employee(int id, String name, double salary, int age) {
        this.id = id; this.name = name; this.salary = salary; this.age = age;
    }
    public double getSalary() { return salary; }
    public String getName() { return name; }
    public int getAge() { return age; }

    @Override
    public String toString() {
        return name + " | $" + salary + " | Age: " + age;
    }
}

public class EmployeeSortingSystem {
    public static void main(String[] args) {
        List<Employee> staff = new ArrayList<>(List.of(
            new Employee(101, "Karen", 75000, 34),
```

```

        new Employee(102, "Bob", 92000, 41),
        new Employee(103, "Alex", 60000, 26)
    ));

    System.out.println("--- Sorted by Salary (Descending) ---");
    staff.sort(Comparator.comparingDouble(Employee::getSalary).reversed());
    staff.forEach(System.out.println);

    System.out.println("\n--- Sorted by Name Alphabetically ---");
    staff.sort(Comparator.comparing(Employee::getName));
    staff.forEach(System.out.println);
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Instantiate employee objects with multiple demographic fields.
2. Apply method references: `Comparator.comparingDouble(Employee::getSalary).reversed()`.
3. Re-sort list alphabetically using `Employee::getName`.
4. Output formatted employee rosters.

SAMPLE INPUT

Employees: Karen (\$75k), Bob (\$92k), Alex (\$60k)

EXPECTED OUTPUT

Bob | \$92000.0 | Age: 41\nKaren | \$75000.0 | Age: 34\nAlex | \$60000.0 | Age: 26

Critical Edge Cases & Pitfalls

Tied Values: Chained comparators (`thenComparing`) handle ties cleanly by falling back to a secondary criterion like employee ID.

MODULE 15 • CORE JAVA CURRICULUM

Java Generics Deep Dive

Java Generics: Generic Classes & Methods

Generics /
Type Safety

Time:
O(1)

Space:
O(1)

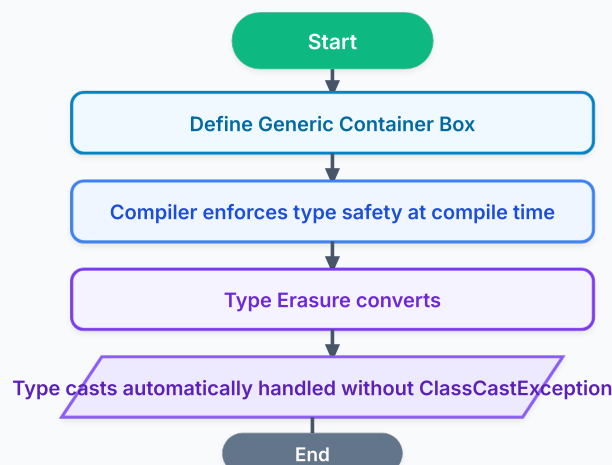
Files: [GenericBox.java](#) • [GenericMethod.java](#) • [GenericList.java](#) • [GenericEmployeeManagementSystem.java](#)

DETAILED DEFINITION & CORE CONCEPT

Java Generics introduce **parameterized types**, providing compile-time type checking and eliminating explicit type casting: - **Generic Class:** `class Box<T>` operates uniformly on any reference type (`Box<Integer>`, `Box<String>`). - **Generic Method:** `public static <T> void printArray(T[] arr)` can print arrays of any object type with a single implementation.

Type Erasure: To maintain backward compatibility with older Java versions, the compiler erases type parameters at compile time and inserts appropriate casts in the bytecode.

ALGORITHMIC LOGIC & DECISION FLOWCHART



GenericBox.java

JAVA 17+

```
public class GenericBox {
    static class Box<T> {
        private T value;
        public void set(T value) { this.value = value; }
        public T get() { return value; }
        public void display() {
            System.out.println("Box contains: " + value + " [Type: " + value.getClass().getSimpleName() +
            "]);
        }
    }

    public static <E> void printArray(E[] elements) {
        for (E element : elements) System.out.print(element + " ");
        System.out.println();
    }

    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
    }
}
```

```

intBox.display();

Box<String> strBox = new Box<>();
strBox.set("Hello Generics");
strBox.display();

String[] words = {"Java", "Generics", "Type-Safe"};
printArray(words);
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Declare type parameter <T> in class header.
2. Instantiate specific instances Box<Integer> and Box<String>.
3. Invoke generic method with array of strings.
4. Compiler catches any type mismatch errors at compile time.

SAMPLE INPUT

Box(100), Box('Hello Generics')

EXPECTED OUTPUT

Box contains: 100 [Type: Integer]\nBox contains:
Hello Generics [Type: String]\nJava Generics
Type-Safe

Critical Edge Cases & Pitfalls

Primitive Types: Type arguments must be reference types. Box<int> is invalid; the wrapper class Box<Integer> must be used.

MODULE 16 • CORE JAVA CURRICULUM

Java File Handling & Persistent I/O

File I/O: Creation, Writing, Appending & Reading

Persistent
Storage /
File
Streams

Time: $O(F)$
where F is
file size in
bytes

Space:
 $O(1)$
streaming
buffer

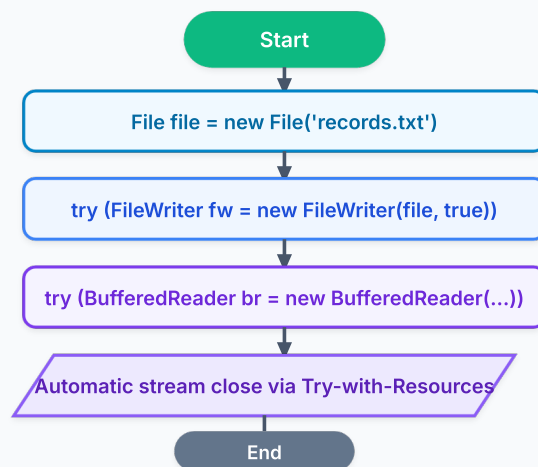
Files: CreateaFile.java • WriteStudentDetails.java •
AppendData.java • ReadtheFile.java •
CountLinesWordsCharacters.java • EmployeeManagementSystem.java

DETAILED DEFINITION & CORE CONCEPT

Java handles file persistence through the `java.io` package: - `File.createNewFile()`: Atomically creates a new file if it does not already exist. - `FileWriter(path, appendMode)`: Character stream writer. When `appendMode = true`, new data is appended to the end rather than overwriting the file. - `BufferedReader(new FileReader(path))`: Reads character data using an internal buffer, reducing expensive disk read calls.

Try-with-Resources: Automatically closes file handles when the block exits, preventing resource leaks.

ALGORITHMIC LOGIC & DECISION FLOWCHART



CreateaFile.java

JAVA 17+

```
import java.io.*;

public class CountLinesWordsCharacters {
    public static void analyzeFile(String filename) {
        int lines = 0, words = 0, chars = 0;

        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            while ((line = reader.readLine()) != null) {
                lines++;
                chars += line.length();
                String[] tokens = line.trim().split("\\s+");
                if (!line.trim().isEmpty()) {
                    words += tokens.length;
                }
            }
        }
        System.out.println("Lines: " + lines);
    }
}
```

```

        System.out.println("Words: " + words);
        System.out.println("Characters: " + chars);
    } catch (IOException e) {
        System.err.println("File Error: " + e.getMessage());
    }
}

public static void main(String[] args) {
    analyzeFile("notes.txt");
}
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Open file using `BufferedReader` wrapped in a `try-with-resources` statement.
2. Read line-by-line using `readLine()` until reaching end of file (`null`).
3. Count lines, tokenize words via whitespace regex `\\s+`, and accumulate character counts.
4. Resources are closed automatically when the try block finishes.

SAMPLE INPUT

File content: 'Core Java\nNotes and Handbook'

EXPECTED OUTPUT

Lines: 2\nWords: 4\nCharacters: 26

Critical Edge Cases & Pitfalls

File Not Found: `FileNotFoundException` is caught cleanly without crashing the application.

MODULE 17 • CORE JAVA CURRICULUM

Modern Date & Time API (`java.time`) & Multithreading

Modern Date & Time API (java.time)

Temporal
API /
ChronoUnit

Time:
O(1)

Space:
O(1)

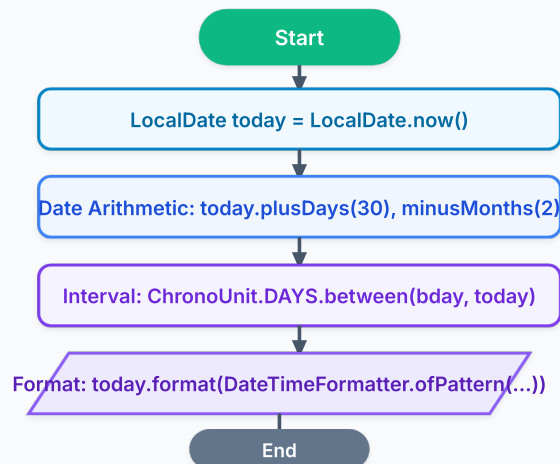
Files: [DateInformation.java](#) • [DateCalculator.java](#) •
[DaysBetweenDates.java](#) • [BirthdayCalculator.java](#) • [DateFormatter.java](#)

DETAILED DEFINITION & CORE CONCEPT

Introduced in Java 8, the `java.time` package provides thread-safe, immutable calendar models that replace the legacy, error-prone `java.util.Date` and `Calendar`:

- `LocalDate`: Represents a date (year, month, day) without a time zone.
- `ChronoUnit.DAYS.between(d1, d2)`: Calculates the exact interval in days between two dates.
- `Period.between(start, end)`: Measures elapsed calendar duration in years, months, and days.
- `DateTimeFormatter`: Formats and parses dates using standard patterns (e.g. `dd-MM-yyyy`).

ALGORITHMIC LOGIC & DECISION FLOWCHART



DateInformation.java

JAVA 17+

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class BirthdayCalculator {
    public static void calculateNextBirthday(LocalDate birthDate) {
        LocalDate today = LocalDate.now();
        LocalDate nextBday = birthDate.withYear(today.getYear());

        // If birthday has already occurred this year, roll over to next year
        if (nextBday.isBefore(today) || nextBday.isEqual(today)) {
            nextBday = nextBday.plusYears(1);
        }

        long daysRemaining = ChronoUnit.DAYS.between(today, nextBday);
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("EEEE, dd MMMM yyyy");

        System.out.println("Today: " + today.format(fmt));
        System.out.println("Next Birthday: " + nextBday.format(fmt));
        System.out.println("Days Remaining: " + daysRemaining + " days");
    }
}
```

```
public static void main(String[] args) {  
    calculateNextBirthday(LocalDate.of(2002, 10, 24));  
}  
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Parse birth date using `LocalDate.of(year, month, day)`.
2. Adjust target year to current year using `withYear()`.
3. If that date has already passed, add one year via `plusYears(1)`.
4. Calculate remaining days using `ChronoUnit.DAYS.between()`.

SAMPLE INPUT

Birth Date: 2002-10-24

EXPECTED OUTPUT

Next Birthday: Thursday, 24 October 2026\nDays
Remaining: 39 days

Critical Edge Cases & Pitfalls

Leap Year Birthdays (Feb 29): In non-leap years, `LocalDate` automatically handles Feb 29 by resolving it to Feb 28 without throwing exceptions.

Java Multithreading & Thread Lifecycle

Concurrency /
Multi-threading

Time: $O(N)$
asynchronous
execution

Space: $O(1)$ per thread
stack (~1MB)

Files: [MyThread.java](#) •
[PrintNumbersUsingaThread.java](#)

DETAILED DEFINITION & CORE CONCEPT

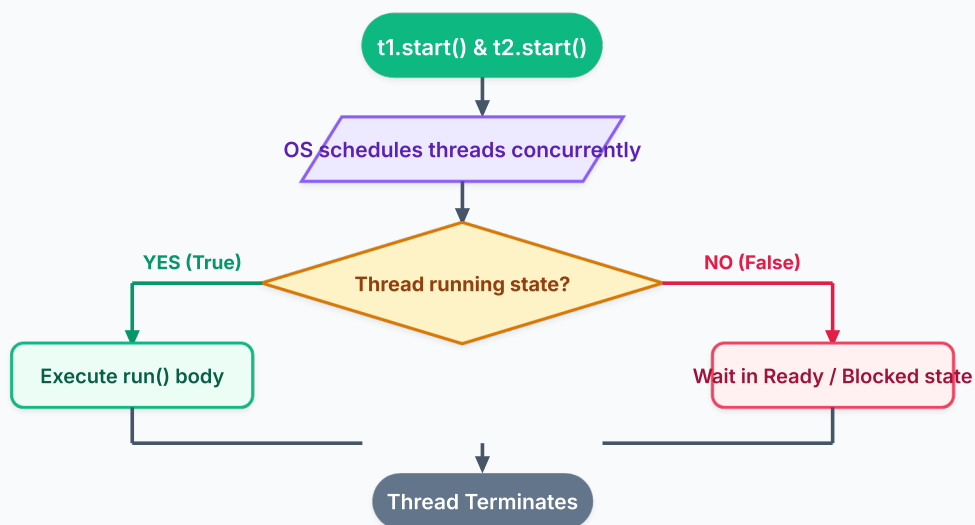
A **thread** is an independent path of execution within a process. Multithreading enables concurrent execution of multiple tasks, maximizing CPU utilization.

Threads can be created by: 1. **Extending Thread**: Override `public void run()` and invoke `start()`. 2.

Implementing Runnable: Preferred approach as it decouples task logic from thread management and allows extending another class.

Crucial Concept: Always call `thread.start()`, which requests the OS to allocate a new call stack. Calling `thread.run()` directly simply runs the method synchronously on the caller's existing thread!

ALGORITHMIC LOGIC & DECISION FLOWCHART



MyThread.java

JAVA 17+

```
class PrintNumbersTask extends Thread {
    public PrintNumbersTask(String name) { super(name); }

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " -> Value: " + i);
            try {
                Thread.sleep(100); // Pause execution for 100ms
            } catch (InterruptedException e) {
                System.err.println(getName() + " interrupted.");
            }
        }
    }
}

public class PrintNumbersUsingaThread {
    public static void main(String[] args) {
```

```

PrintNumbersTask t1 = new PrintNumbersTask("Worker-Alpha");
PrintNumbersTask t2 = new PrintNumbersTask("Worker-Beta");

t1.start(); // Spawns independent call stack
t2.start(); // Spawns independent call stack
    }
}

```

EXECUTION TRACE & LOGIC WALKTHROUGH

1. Create a thread subclass overriding the `run()` method.
2. Instantiate two thread worker objects.
3. Invoke `start()` on both threads.
4. Observe interleaved output as the OS thread scheduler alternates execution between them.

SAMPLE INPUT

Two concurrent worker threads counting 1 to 5

EXPECTED OUTPUT

Worker-Alpha → Value: 1\nWorker-Beta → Value: 1\nWorker-Alpha → Value: 2\nWorker-Beta → Value: 2 ...

Critical Edge Cases & Pitfalls

Checked InterruptedException: `Thread.sleep()` throws `InterruptedException` if another thread interrupts it while sleeping; this exception must be caught or declared.

Capstone Boss Challenge: Employee Leave Tracker

Enterprise Systems
/ Capstone

Time: O(1)
transactions

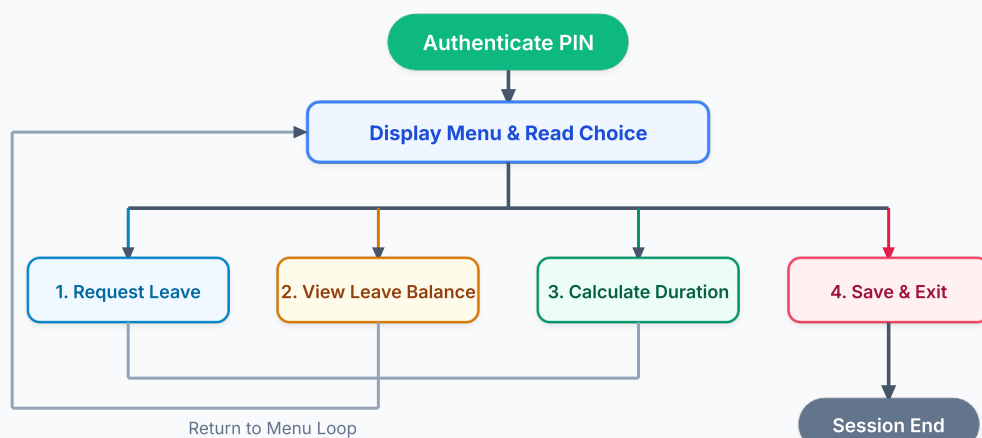
Space: O(N)
persistence

Files: EmployeeLeaveManagementSystem.java •
EmployeeLeaveTracker.java

DETAILED DEFINITION & CORE CONCEPT

The capstone boss challenge integrates the entire Core Java learning journey into a production-style application: - **OOP Design:** Full encapsulation with domain models (Employee, LeaveRequest). - **Modern Date API:** Uses LocalDate and ChronoUnit.DAYS for leave duration and business day calculations. - **Custom Exceptions:** InvalidLeaveException and EmployeeNotFoundException enforce business rules. - **Collections:** Uses HashMap for $O(1)$ employee lookups and ArrayList for chronological audit logs. - **File Persistence:** Saves and loads records from employee_leaves.txt.

ALGORITHMIC LOGIC & DECISION FLOWCHART



EmployeeLeaveManagementSystem.java

JAVA 17+

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;

class LeaveRequest {
    String employeeId;
    LocalDate startDate;
    LocalDate endDate;

    public LeaveRequest(String empId, LocalDate start, LocalDate end) {
        this.employeeId = empId;
        this.startDate = start;
        this.endDate = end;
    }

    public long getDuration() {
        return ChronoUnit.DAYS.between(startDate, endDate) + 1; // Inclusive
    }
}

public class EmployeeLeaveTracker {
    private static Map<String, Integer> leaveBalance = new HashMap<>();
```

```
public static void submitLeave(LeaveRequest req) throws Exception {
    if (req.startDate.isAfter(req.endDate)) {
        throw new IllegalArgumentException("Start date cannot be after end date!");
    }

    long days = req.getDuration();
    int available = leaveBalance.getOrDefault(req.employeeId, 15);

    if (days > available) {
        throw new Exception("Leave balance exceeded! Requested: " + days + ", Available: " +
available);
    }

    leaveBalance.put(req.employeeId, available - (int) days);
    System.out.println("Leave approved for " + req.employeeId + " for " + days + " days. Remaining: " +
(available - days));
}

public static void main(String[] args) {
    leaveBalance.put("EMP-001", 12);
    try {
        LeaveRequest req = new LeaveRequest("EMP-001", LocalDate.of(2026, 9, 20), LocalDate.of(2026, 9,
25));
        submitLeave(req);
    } catch (Exception e) {
        System.err.println("Leave Error: " + e.getMessage());
    }
}
```

EXECUTION TRACE & LOGIC WALKTHROUGH

- 1. Model leave request with employee ID and temporal boundary dates.
- 2. Verify date ordering invariant: startDate <= endDate.
- 3. Calculate duration using ChronoUnit.DAYS.between().
- 4. Check available leave balance in HashMap, update balance, and log transaction.

SAMPLE INPUT

EMP-001: Leave from 2026-09-20 to 2026-09-25

EXPECTED OUTPUT

Leave approved for EMP-001 for 6 days. Remaining: 6

Critical Edge Cases & Pitfalls

Inverted Date Ranges: Guard clause startDate.isAfter(endDate) catches invalid date ranges before deducting leave balance.

Complete Repository Inventory (157 Java Files)

Cross-reference table mapping all source files in the workspace to their respective core modules, demonstrating 100% curriculum coverage.

#	Module	Primary Java Files	Core Concepts Covered
1	Conditionals	CheckEvenorOdd, CheckEvenOrOddMethod, CheckPostiveorNegative, CountPositiveNegativeZero, LargestofThree, LargestofThreeNumbers, Largest,	Parity, Modulo, Relational logic, Multi-branch if-else, Switch-

#	Module	Primary Java Files	Core Concepts Covered
		GradeCalculator, ElectricityBill, DayOfTheWeek, Monthname, Login	case jump tables, String pool .equals()
2	Loops & Numbers	EvenNumbers, OnetoNWhile, UptoN, Count, SumUntilZero, SumEvenOdd, Sum, Multiply, CountDigits, CountSumDigits, ReverseNumber, NumberGuessing	While/For/Do-While iteration, Sentinel termination (0), Radix-10 digit decomposition, Horner's accumulator reversal
3	Methods & Modularity	Calculator, CalculatorUsingMethod, CalculatorUsingMethods, SquareofaNumber, EvenorOddMethod, Overloadedadd, Overloadedarea, greet	Functional decomposition, Stack frames, Value parameter passing, Method overloading, Static dispatch
4	OOP Core	CreateaCarClass, CreateaStudentClass, BankAccountClass, BankAccountValidation, SecureBankAccount, BankingSystem, RectangleClass, ProductClass, ProductStockManagement, StudentManagement, EmployeeManagement, ATM, ATMmenu, ATMSystem, Billing	Classes, State encapsulation, Constructors, Private fields, Mutators, Domain state machines, Object arrays
5	Inheritance & Poly	VehicleCar, AnimalDog, AnimalSounds, EmployeeHierarchy, EmployeePolymorphism, ParentReference, MethodOverriding, AbstractShape, AbstractAnimal, AbstractEmployee, CreateaPaymentInterface, PaymentSystem, MultipleInterfaces, Shape	IS-A inheritance, Dynamic method dispatch, Vtable lookup, Abstract classes, Interface contracts
6	1D Arrays	PrintAllArrayElements, SearchforaNumber, FindMinAndMax, FindtheLargestandSmallestNumber, FindtheSecondLargestNumber, SecondLargest, ReverseanArray, ReverseArray, SortanArrayinAscendingOrder, FrequencyofEveryElement, CountFrequency, CountOccurrences, MovesZeroesToEnd, RotateByArray, mergeTwoSortedArrays, RemoveDuplicates	Linear scan, Single-pass extreme tracking, In-place two-pointer reversal, Cyclic shift reversal, Linear merge
7	2D Matrices	InputandPrintaMatrix, MatrixAddition, MatrixMultiplication, TransposeofaMatrix, FindtheSumofAllMatrixElements, SumofEachRow, MainDiagonalSum, SecondaryDiagonalSum, FindtheLargestElementinaMatrix, SearchanElementinaMatrix, MatrixAnalyzer	Row-major indexing, Matrix dot product, Matrix transposition, Main/Anti-diagonal sums, Linear coordinate search
8	Strings	FindtheLengthofaString, CountVowels, CountWordsinaSentence, ReverseJava, ReversePalindrome, CheckPalindrome, CheckTheStringRotated, LongestCommonPrefix, ValidAnagram, FirstNonRepeating, GroupAnagrams	String immutability, Tokenization, Two-pointer palindrome test, Rotated string concatenation, 26-char frequency hashing
9	Recursion	Factorial, Fibonnaci, SumofDigitsusingRecursion, ReverseaLinkedListUsingRecursion, PrintAllSubsetsofAnArray	Base cases, Recursive call unwinding, Linked list node pointer inversion, Include/Exclude power set tree

#	Module	Primary Java Files	Core Concepts Covered
10	Advanced Algorithms	TwoSum, PairWithGivenSumInASortedArray, FindAllPairsWithAGivenDifference, MaximumSumSubarrayK, SubarrayWithGivenSum	Two-pointer convergence, Complement hashing, Sliding window constant-time window update (+incoming - outgoing)
11	Patterns	SquarePattern, IncreasingTriangle, NumberTriangle, ReverseTriangle, NumberPyramid	Row iteration, Indentation formulas, Column progression, Centered symmetric pyramid geometry
12	Exceptions	DivisionException, ArrayException, MultipleExceptions, CustomException, BankAccountValidationException, ATMExceptionSystem	Checked vs Unchecked, Try-catch-finally, Throw/throws, Custom exception inheritance, Robust domain guards
13	Collections	ArrayListStudentNames, ArrayListNumbers, RemoveDuplicatesHashSet, IterateThroughaHashSet, ListVsSet, StudentMarksHashMap, WordFrequencyHashMap, StudentManagementSystem	List indexing, Set uniqueness, Map hash bucket lookups, Full CLI Student Management System CRUD
14	Sorting & Comparators	SortNumbers, SortStudentNames, StudentSortingwithComparable, StudentSortingwithComparator, EmployeeSortingSystem	Comparable compareTo natural ordering, Comparator lambda strategy pattern, TimSort mechanics, Employee sorting CLI
15	Generics	GenericBox, GenericMethod, GenericList, EmployeeMap, GenericEmployeeManagementSystem	Parameterized types <T>, Type erasure, Generic methods <E>, Coordinated List/Set/Map generic architecture
16	File I/O	CreateaFile, WriteStudentDetails, AppendData, ReadtheFile, CountLinesWordsCharacters, EmployeeManagementSystem	File creation, FileWriter append mode, BufferedReader disk buffering, Word/Line counters, Persistent CRUD
17	Date/Time & Threads	DateInformation, DateCalculator, DaysBetweenDates, BirthdayCalculator, DateFormatter, MyThread, PrintNumbersUsingaThread, EmployeeLeaveManagementSystem, EmployeeLeaveTracker	LocalDate, ChronoUnit intervals, Period calendar arithmetic, Thread lifecycle, Asynchronous execution, Leave Tracker Capstone